Add these lines:

```
dnl set SASL options
TRUST_AUTH_MECH(`GSSAPI DIGEST-MD5 CRAM-MD5 LOGIN')dnl
define(`confAUTH_MECHANISMS', `GSSAPI DIGEST-MD5 CRAM-MD5 LOGIN')dnl
```

These options configure the different methods available to Sendmail for authenticating users. To use a method other than pwcheck, refer to the Sendmail documentation.

Finally, run make(1) while in /etc/mail. That will run the new .mc and create a .cf named either freebsd.cf or the name used for the local .mc.

Then, run `make install restart`, which will copy the file to sendmail.cf, and properly restart Sendmail.

For more information about this process, refer to /etc/mail/Makefile.

To test the configuration, use a MUA to send a test message. For further investigation, set the `LogLevel` of Sendmail to `13` and watch /var/log/maillog for any errors.

For more information, refer to SMTP authentication.

# Chapter 32. Network Servers

## 32.1. Synopsis

This chapter covers some of the more frequently used network services on UNIX® systems. This includes installing, configuring, testing, and maintaining many different types of network services. Example configuration files are included throughout this chapter for reference.

By the end of this chapter, readers will know:

- How to manage the inetd daemon.
- How to set up the Network File System (NFS).
- How to set up the Network Information Server (NIS) for centralizing and sharing user accounts.
- How to set FreeBSD up to act as an LDAP server or client
- How to set up automatic network settings using DHCP.
- How to set up a Domain Name Server (DNS).
- How to set up the Apache HTTP Server.
- How to set up a File Transfer Protocol (FTP) server.
- How to set up a file and print server for Windows® clients using Samba.
- How to synchronize the time and date, and set up a time server using the Network Time Protocol (NTP).
- How to set up iSCSI.

This chapter assumes a basic knowledge of:

- /etc/rc scripts.
- Network terminology.
- Installation of additional third-party software (Installing Applications: Packages and Ports).

## 32.2. The inetd Super-Server

The inetd(8) daemon is sometimes referred to as a Super-Server because it manages connections for many services. Instead of starting multiple applications, only the inetd service needs to be started. When a connection is received for a service that is managed by inetd, it determines which program the connection is destined for, spawns a process for that program, and delegates the program a socket. Using inetd for services that are not heavily used can reduce system load, when compared to running each daemon individually in stand-alone mode.

Primarily, inetd is used to spawn other daemons, but several trivial protocols are handled internally, such as chargen, auth, time, echo, discard, and daytime.

This section covers the basics of configuring inetd.

### 32.2.1. Configuration File

Configuration of inetd is done by editing /etc/inetd.conf. Each line of this configuration file represents an application which can be started by inetd. By default, every line starts with a comment (#), meaning that inetd is not listening for any applications. To configure inetd to listen for an application's connections, remove the # at the beginning of the line for that application.

After saving the edits, configure inetd to start at system boot by editing /etc/rc.conf:

```
inetd_enable="YES"
```

To start inetd now, so that it listens for the configured service, type:

```
# service inetd start
```

Once inetd is started, it needs to be notified whenever a modification is made to /etc/inetd.conf:

*Example 41. Reloading the inetd Configuration File*

```
# service inetd reload
```

Typically, the default entry for an application does not need to be edited beyond removing the #. In some situations, it may be appropriate to edit the default entry.

As an example, this is the default entry for ftpd(8) over IPv4:

```
ftp     stream  tcp     nowait  root    /usr/libexec/ftpd       ftpd -l
```

The seven columns in an entry are as follows:

```
service-name
socket-type
protocol
{wait|nowait}[/max-child[/max-connections-per-ip-per-minute[/max-child-per-ip]]]
user[:group][/login-class]
server-program
server-program-arguments
```

where:

**service-name**

The service name of the daemon to start. It must correspond to a service listed in /etc/services. This determines which port inetd listens on for incoming connections to that service. When using a custom service, it must first be added to /etc/services.

**socket-type**

Either `stream`, `dgram`, `raw`, or `seqpacket`. Use `stream` for TCP connections and `dgram` for UDP services.

**protocol**

Use one of the following protocol names:

| Protocol Name | Explanation |
|---|---|
| tcp or tcp4 | TCP IPv4 |
| udp or udp4 | UDP IPv4 |
| tcp6 | TCP IPv6 |
| udp6 | UDP IPv6 |
| tcp46 | Both TCP IPv4 and IPv6 |
| udp46 | Both UDP IPv4 and IPv6 |

**{wait|nowait}[/max-child[/max-connections-per-ip-per-minute[/max-child-per-ip]]]**

In this field, `wait` or `nowait` must be specified. `max-child`, `max-connections-per-ip-per-minute` and `max-child-per-ip` are optional.

`wait|nowait` indicates whether or not the service is able to handle its own socket. `dgram` socket types must use `wait` while `stream` daemons, which are usually multi-threaded, should use `nowait`. `wait` usually hands off multiple sockets to a single daemon, while `nowait` spawns a child daemon for each new socket.

The maximum number of child daemons inetd may spawn is set by `max-child`. For example, to limit ten instances of the daemon, place a `/10` after `nowait`. Specifying `/0` allows an unlimited number of children.

`max-connections-per-ip-per-minute` limits the number of connections from any particular IP address per minute. Once the limit is reached, further connections from this IP address will be dropped until the end of the minute. For example, a value of `/10` would limit any particular IP address to ten connection attempts per minute. `max-child-per-ip` limits the number of child processes that can be started on behalf on any single IP address at any moment. These options can limit excessive resource consumption and help to prevent Denial of Service attacks.

An example can be seen in the default settings for fingerd(8):

```
finger stream  tcp     nowait/3/10 nobody /usr/libexec/fingerd fingerd -k -s
```

**user**

The username the daemon will run as. Daemons typically run as `root`, `daemon`, or `nobody`.

**server-program**

The full path to the daemon. If the daemon is a service provided by inetd internally, use `internal`.

**server-program-arguments**

Used to specify any command arguments to be passed to the daemon on invocation. If the daemon is an internal service, use `internal`.

### 32.2.2. Command-Line Options

Like most server daemons, inetd has a number of options that can be used to modify its behavior. By default, inetd is started with `-wW -C 60`. These options enable TCP wrappers for all services, including internal services, and prevent any IP address from requesting any service more than 60 times per minute.

To change the default options which are passed to inetd, add an entry for `inetd_flags` in /etc/rc.conf. If inetd is already running, restart it with `service inetd restart`.

The available rate limiting options are:

**-c maximum**

Specify the default maximum number of simultaneous invocations of each service, where the default is unlimited. May be overridden on a per-service basis by using `max-child` in /etc/inetd.conf.

**-C rate**

Specify the default maximum number of times a service can be invoked from a single IP address per minute. May be overridden on a per-service basis by using `max-connections-per-ip-per-minute` in /etc/inetd.conf.

**-R rate**

Specify the maximum number of times a service can be invoked in one minute, where the default is `256`. A rate of `0` allows an unlimited number.

**-s maximum**

Specify the maximum number of times a service can be invoked from a single IP address at any one time, where the default is unlimited. May be overridden on a per-service basis by using `max-child-per-ip` in /etc/inetd.conf.

Additional options are available. Refer to inetd(8) for the full list of options.

### 32.2.3. Security Considerations

Many of the daemons which can be managed by inetd are not security-conscious. Some daemons, such as fingerd, can provide information that may be useful to an attacker. Only enable the services which are needed and monitor the system for excessive connection attempts. `max-connections-per-ip-per-minute`, `max-child` and `max-child-per-ip` can be used to limit such attacks.

By default, TCP wrappers are enabled. Consult hosts_access(5) for more information on placing TCP restrictions on various inetd invoked daemons.

## 32.3. Network File System (NFS)

FreeBSD supports the Network File System (NFS), which allows a server to share directories and files with clients over a network. With NFS, users and programs can access files on remote systems as if they were stored locally.

NFS has many practical uses. Some of the more common uses include:

- Data that would otherwise be duplicated on each client can be kept in a single location and accessed by clients on the network.
- Several clients may need access to the /usr/ports/distfiles directory. Sharing that directory allows for quick access to the source files without having to download them to each client.
- On large networks, it is often more convenient to configure a central NFS server on which all user home directories are stored. Users can log into a client anywhere on the network and have access to their home directories.
- Administration of NFS exports is simplified. For example, there is only one file system where security or backup policies must be set.
- Removable media storage devices can be used by other machines on the network. This reduces the number of devices throughout the network and provides a centralized location to manage their security. It is often more convenient to install software on multiple machines from a centralized installation media.

NFS consists of a server and one or more clients. The client remotely accesses the data that is stored on the server machine. In order for this to function properly, a few processes have to be configured and running.

These daemons must be running on the server:

| Daemon | Description |
|---|---|
| nfsd | The NFS daemon which services requests from NFS clients. |
| mountd | The NFS mount daemon which carries out requests received from nfsd. |
| rpcbind | This daemon allows NFS clients to discover which port the NFS server is using. |

Running nfsiod(8) on the client can improve performance, but is not required.

### 32.3.1. Configuring the Server

The file systems which the NFS server will share are specified in /etc/exports. Each line in this file specifies a file system to be exported, which clients have access to that file system, and any access options. When adding entries to this file, each exported file system, its properties, and allowed hosts must occur on a single line. If no clients are listed in the entry, then any client on the network can mount that file system.

The following /etc/exports entries demonstrate how to export file systems. The examples can be modified to match the file systems and client names on the reader's network. There are many options that can be used in this file, but only a few will be mentioned here. See exports(5) for the full list of options.

This example shows how to export /media to three hosts named *alpha*, *bravo*, and *charlie*:

```
/media -ro alpha bravo charlie
```

The `-ro` flag makes the file system read-only, preventing clients from making any changes to the exported file system. This example assumes that the host names are either in DNS or in /etc/hosts. Refer to hosts(5) if the network does not have a DNS server.

The next example exports /home to three clients by IP address. This can be useful for networks without DNS or /etc/hosts entries. The `-alldirs` flag allows subdirectories to be mount points. In other words, it will not automatically mount the subdirectories, but will permit the client to mount the directories that are required as needed.

```
/usr/home  -alldirs  10.0.0.2 10.0.0.3 10.0.0.4
```

This next example exports /a so that two clients from different domains may access that file system. The `-maproot=root` allows `root` on the remote system to write data on the exported file system as `root`. If `-maproot=root` is not specified, the client's `root` user will be mapped to the server's `nobody` account and will be subject to the access limitations defined for `nobody`.

```
/a  -maproot=root  host.example.com box.example.org
```

A client can only be specified once per file system. For example, if /usr is a single file system, these entries would be invalid as both entries specify the same host:

```
# Invalid when /usr is one file system
/usr/src   client
/usr/ports client
```

The correct format for this situation is to use one entry:

```
/usr/src /usr/ports  client
```

The following is an example of a valid export list, where /usr and /exports are local file systems:

```
# Export src and ports to client01 and client02, but only
# client01 has root privileges on it
/usr/src /usr/ports -maproot=root    client01
/usr/src /usr/ports               client02
```

```
# The client machines have root and can mount anywhere
# on /exports. Anyone in the world can mount /exports/obj read-only
/exports -alldirs -maproot=root      client01 client02
/exports/obj -ro
```

To enable the processes required by the NFS server at boot time, add these options to /etc/rc.conf:

```
rpcbind_enable="YES"
nfs_server_enable="YES"
mountd_enable="YES"
```

The server can be started now by running this command:

```
# service nfsd start
```

Whenever the NFS server is started, mountd also starts automatically. However, mountd only reads /etc/exports when it is started. To make subsequent /etc/exports edits take effect immediately, force mountd to reread it:

```
# service mountd reload
```

Refer to zfs-share(8) for a description of exporting ZFS datasets via NFS using the `sharenfs` ZFS property instead of the exports(5) file.

Refer to nfsv4(4) for a description of an NFS Version 4 setup.

### 32.3.2. Configuring the Client

To enable NFS clients, set this option in each client's /etc/rc.conf:

```
nfs_client_enable="YES"
```

Then, run this command on each NFS client:

```
# service nfsclient start
```

The client now has everything it needs to mount a remote file system. In these examples, the server's name is `server` and the client's name is `client`. To mount /home on `server` to the /mnt mount point on `client`:

```
# mount server:/home /mnt
```

The files and directories in /home will now be available on `client`, in the /mnt directory.

To mount a remote file system each time the client boots, add it to /etc/fstab:

```
server:/home    /mnt    nfs rw  0   0
```

Refer to fstab(5) for a description of all available options.

### 32.3.3. Locking

Some applications require file locking to operate correctly. To enable locking, execute the following command on both the client and server:

```
# sysrc rpc_lockd_enable="YES"
```

Then start the rpc.lockd(8) service:

```
# service lockd start
```

If locking is not required on the server, the NFS client can be configured to lock locally by including `-L` when running mount. Refer to mount_nfs(8) for further details.

### 32.3.4. Automating Mounts with autofs(5)



The autofs(5) automount facility is supported starting with FreeBSD 10.1-RELEASE. To use the automounter functionality in older versions of FreeBSD, use amd(8) instead. This chapter only describes the autofs(5) automounter.

The autofs(5) facility is a common name for several components that, together, allow for automatic mounting of remote and local filesystems whenever a file or directory within that file system is accessed. It consists of the kernel component, autofs(5), and several userspace applications: automount(8), automountd(8) and autounmountd(8). It serves as an alternative for amd(8) from previous FreeBSD releases. amd is still provided for backward compatibility purposes, as the two use different map formats; the one used by autofs is the same as with other SVR4 automounters, such as the ones in Solaris, MacOS X, and Linux.

The autofs(5) virtual filesystem is mounted on specified mountpoints by automount(8), usually invoked during boot.

Whenever a process attempts to access a file within the autofs(5) mountpoint, the kernel will notify automountd(8) daemon and pause the triggering process. The automountd(8) daemon will handle kernel requests by finding the proper map and mounting the filesystem according to it, then signal the kernel to release blocked process. The autounmountd(8) daemon automatically unmounts automounted filesystems after some time, unless they are still being used.

The primary autofs configuration file is /etc/auto_master. It assigns individual maps to top-level mounts. For an explanation of auto_master and the map syntax, refer to auto_master(5).

There is a special automounter map mounted on /net. When a file is accessed within this directory, autofs(5) looks up the corresponding remote mount and automatically mounts it. For instance, an attempt to access a file within /net/foobar/usr would tell automountd(8) to mount the /usr export from the host `foobar`.

*Example 42. Mounting an Export with autofs(5)*

In this example, `showmount -e` shows the exported file systems that can be mounted from the NFS server, `foobar`:

```
% showmount -e foobar
Exports list on foobar:
/usr                                10.10.10.0
/a                                  10.10.10.0
% cd /net/foobar/usr
```

The output from `showmount` shows /usr as an export. When changing directories to /host/foobar/usr, automountd(8) intercepts the request and attempts to resolve the hostname `foobar`. If successful, automountd(8) automatically mounts the source export.

To enable autofs(5) at boot time, add this line to /etc/rc.conf:

```
autofs_enable="YES"
```

Then autofs(5) can be started by running:

```
# service automount start
# service automountd start
# service autounmountd start
```

The autofs(5) map format is the same as in other operating systems. Information about this format from other sources can be useful, like the Mac OS X document.

Consult the automount(8), automountd(8), autounmountd(8), and auto_master(5) manual pages for more information.

# 32.4. Network Information System (NIS)

Network Information System (NIS) is designed to centralize administration of UNIX®-like systems such as Solaris™, HP-UX, AIX®, Linux, NetBSD, OpenBSD, and FreeBSD. NIS was originally known as Yellow Pages but the name was changed due to trademark issues. This is the reason why NIS commands begin with `yp`.

NIS is a Remote Procedure Call (RPC)-based client/server system that allows a group of machines within an NIS domain to share a common set of configuration files. This permits a system

administrator to set up NIS client systems with only minimal configuration data and to add, remove, or modify configuration data from a single location.

FreeBSD uses version 2 of the NIS protocol.

### 32.4.1. NIS Terms and Processes

Table 28.1 summarizes the terms and important processes used by NIS:

*Table 45. NIS Terminology*

| Term | Description |
|---|---|
| NIS domain name | NIS servers and clients share an NIS domain name. Typically, this name does not have anything to do with DNS. |
| rpcbind(8) | This service enables RPC and must be running in order to run an NIS server or act as an NIS client. |
| ypbind(8) | This service binds an NIS client to its NIS server. It will take the NIS domain name and use RPC to connect to the server. It is the core of client/server communication in an NIS environment. If this service is not running on a client machine, it will not be able to access the NIS server. |
| ypserv(8) | This is the process for the NIS server. If this service stops running, the server will no longer be able to respond to NIS requests so hopefully, there is a slave server to take over. Some non-FreeBSD clients will not try to reconnect using a slave server and the ypbind process may need to be restarted on these clients. |
| rpc.yppasswdd(8) | This process only runs on NIS master servers. This daemon allows NIS clients to change their NIS passwords. If this daemon is not running, users will have to login to the NIS master server and change their passwords there. |

### 32.4.2. Machine Types

There are three types of hosts in an NIS environment:

- NIS master server

  This server acts as a central repository for host configuration information and maintains the authoritative copy of the files used by all of the NIS clients. The passwd, group, and other various files used by NIS clients are stored on the master server. While it is possible for one

machine to be an NIS master server for more than one NIS domain, this type of configuration will not be covered in this chapter as it assumes a relatively small-scale NIS environment.

- NIS slave servers

  NIS slave servers maintain copies of the NIS master's data files in order to provide redundancy. Slave servers also help to balance the load of the master server as NIS clients always attach to the NIS server which responds first.

- NIS clients

  NIS clients authenticate against the NIS server during log on.

Information in many files can be shared using NIS. The master.passwd, group, and hosts files are commonly shared via NIS. Whenever a process on a client needs information that would normally be found in these files locally, it makes a query to the NIS server that it is bound to instead.

### 32.4.3. Planning Considerations

This section describes a sample NIS environment which consists of 15 FreeBSD machines with no centralized point of administration. Each machine has its own /etc/passwd and /etc/master.passwd. These files are kept in sync with each other only through manual intervention. Currently, when a user is added to the lab, the process must be repeated on all 15 machines.

The configuration of the lab will be as follows:

| Machine name | IP address | Machine role |
|---|---|---|
| `ellington` | `10.0.0.2` | NIS master |
| `coltrane` | `10.0.0.3` | NIS slave |
| `basie` | `10.0.0.4` | Faculty workstation |
| `bird` | `10.0.0.5` | Client machine |
| `cli[1-11]` | `10.0.0.[6-17]` | Other client machines |

If this is the first time an NIS scheme is being developed, it should be thoroughly planned ahead of time. Regardless of network size, several decisions need to be made as part of the planning process.

#### 32.4.3.1. Choosing a NIS Domain Name

When a client broadcasts its requests for info, it includes the name of the NIS domain that it is part of. This is how multiple servers on one network can tell which server should answer which request. Think of the NIS domain name as the name for a group of hosts.

Some organizations choose to use their Internet domain name for their NIS domain name. This is not recommended as it can cause confusion when trying to debug network problems. The NIS domain name should be unique within the network and it is helpful if it describes the group of machines it represents. For example, the Art department at Acme Inc. might be in the "acme-art" NIS domain. This example will use the domain name `test-domain`.

However, some non-FreeBSD operating systems require the NIS domain name to be the same as the Internet domain name. If one or more machines on the network have this restriction, the Internet domain name *must* be used as the NIS domain name.

#### 32.4.3.2. Physical Server Requirements

There are several things to keep in mind when choosing a machine to use as a NIS server. Since NIS clients depend upon the availability of the server, choose a machine that is not rebooted frequently. The NIS server should ideally be a stand alone machine whose sole purpose is to be an NIS server. If the network is not heavily used, it is acceptable to put the NIS server on a machine running other services. However, if the NIS server becomes unavailable, it will adversely affect all NIS clients.

### 32.4.4. Configuring the NIS Master Server

The canonical copies of all NIS files are stored on the master server. The databases used to store the information are called NIS maps. In FreeBSD, these maps are stored in /var/yp/[domainname] where [domainname] is the name of the NIS domain. Since multiple domains are supported, it is possible to have several directories, one for each domain. Each domain will have its own independent set of maps.

NIS master and slave servers handle all NIS requests through ypserv(8). This daemon is responsible for receiving incoming requests from NIS clients, translating the requested domain and map name to a path to the corresponding database file, and transmitting data from the database back to the client.

Setting up a master NIS server can be relatively straight forward, depending on environmental needs. Since FreeBSD provides built-in NIS support, it only needs to be enabled by adding the following lines to /etc/rc.conf:

```
nisdomainname="test-domain" ①
nis_server_enable="YES"     ②
nis_yppasswdd_enable="YES"  ③
```

① This line sets the NIS domain name to `test-domain`.

② This automates the start up of the NIS server processes when the system boots.

③ This enables the rpc.yppasswdd(8) daemon so that users can change their NIS password from a client machine.

Care must be taken in a multi-server domain where the server machines are also NIS clients. It is generally a good idea to force the servers to bind to themselves rather than allowing them to broadcast bind requests and possibly become bound to each other. Strange failure modes can result if one server goes down and others are dependent upon it. Eventually, all the clients will time out and attempt to bind to other servers, but the delay involved can be considerable and the failure mode is still present since the servers might bind to each other all over again.

A server that is also a client can be forced to bind to a particular server by adding these additional lines to /etc/rc.conf:

```
nis_client_enable="YES"             ①
nis_client_flags="-S test-domain,server"    ②
```

① This enables running client stuff as well.

② This line sets the NIS domain name to `test-domain` and bind to itself.

After saving the edits, type `/etc/netstart` to restart the network and apply the values defined in /etc/rc.conf. Before initializing the NIS maps, start ypserv(8):

```
# service ypserv start
```

#### 32.4.4.1. Initializing the NIS Maps

NIS maps are generated from the configuration files in /etc on the NIS master, with one exception: /etc/master.passwd. This is to prevent the propagation of passwords to all the servers in the NIS domain. Therefore, before the NIS maps are initialized, configure the primary password files:

```
# cp /etc/master.passwd /var/yp/master.passwd
# cd /var/yp
# vi master.passwd
```

It is advisable to remove all entries for system accounts as well as any user accounts that do not need to be propagated to the NIS clients, such as the `root` and any other administrative accounts.



Ensure that the /var/yp/master.passwd is neither group or world readable by setting its permissions to `600`.

After completing this task, initialize the NIS maps. FreeBSD includes the ypinit(8) script to do this. When generating maps for the master server, include `-m` and specify the NIS domain name:

```
ellington# ypinit -m test-domain
Server Type: MASTER Domain: test-domain
Creating an YP server will require that you answer a few questions.
Questions will all be asked at the beginning of the procedure.
Do you want this procedure to quit on non-fatal errors? [y/n: n] n
Ok, please remember to go back and redo manually whatever fails.
If not, something might not work.
At this point, we have to construct a list of this domains YP servers.
rod.darktech.org is already known as master server.
Please continue to add any slave servers, one per line. When you are
done with the list, type a <control D>.
master server   :  ellington
next host to add:  coltrane
next host to add:  ^D
The current list of NIS servers looks like this:
ellington
```

```
coltrane
Is this correct?  [y/n: y] y

[..output from map generation..]

NIS Map update completed.
ellington has been setup as an YP master server without any errors.
```

This will create /var/yp/Makefile from /var/yp/Makefile.dist. By default, this file assumes that the environment has a single NIS server with only FreeBSD clients. Since `test-domain` has a slave server, edit this line in /var/yp/Makefile so that it begins with a comment (`#`):

```
NOPUSH = "True"
```

#### 32.4.4.2. Adding New Users

Every time a new user is created, the user account must be added to the master NIS server and the NIS maps rebuilt. Until this occurs, the new user will not be able to login anywhere except on the NIS master. For example, to add the new user `jsmith` to the `test-domain` domain, run these commands on the master server:

```
# pw useradd jsmith
# cd /var/yp
# make test-domain
```

The user could also be added using `adduser jsmith` instead of `pw useradd smith`.

### 32.4.5. Setting up a NIS Slave Server

To set up an NIS slave server, log on to the slave server and edit /etc/rc.conf as for the master server. Do not generate any NIS maps, as these already exist on the master server. When running `ypinit` on the slave server, use `-s` (for slave) instead of `-m` (for master). This option requires the name of the NIS master in addition to the domain name, as seen in this example:

```
coltrane# ypinit -s ellington test-domain

Server Type: SLAVE Domain: test-domain Master: ellington

Creating an YP server will require that you answer a few questions.
Questions will all be asked at the beginning of the procedure.

Do you want this procedure to quit on non-fatal errors? [y/n: n]  n

Ok, please remember to go back and redo manually whatever fails.
If not, something might not work.
There will be no further questions. The remainder of the procedure
should take a few minutes, to copy the databases from ellington.
```

```
Transferring netgroup...
ypxfr: Exiting: Map successfully transferred
Transferring netgroup.byuser...
ypxfr: Exiting: Map successfully transferred
Transferring netgroup.byhost...
ypxfr: Exiting: Map successfully transferred
Transferring master.passwd.byuid...
ypxfr: Exiting: Map successfully transferred
Transferring passwd.byuid...
ypxfr: Exiting: Map successfully transferred
Transferring passwd.byname...
ypxfr: Exiting: Map successfully transferred
Transferring group.bygid...
ypxfr: Exiting: Map successfully transferred
Transferring group.byname...
ypxfr: Exiting: Map successfully transferred
Transferring services.byname...
ypxfr: Exiting: Map successfully transferred
Transferring rpc.bynumber...
ypxfr: Exiting: Map successfully transferred
Transferring rpc.byname...
ypxfr: Exiting: Map successfully transferred
Transferring protocols.byname...
ypxfr: Exiting: Map successfully transferred
Transferring master.passwd.byname...
ypxfr: Exiting: Map successfully transferred
Transferring networks.byname...
ypxfr: Exiting: Map successfully transferred
Transferring networks.byaddr...
ypxfr: Exiting: Map successfully transferred
Transferring netid.byname...
ypxfr: Exiting: Map successfully transferred
Transferring hosts.byaddr...
ypxfr: Exiting: Map successfully transferred
Transferring protocols.bynumber...
ypxfr: Exiting: Map successfully transferred
Transferring ypservers...
ypxfr: Exiting: Map successfully transferred
Transferring hosts.byname...
ypxfr: Exiting: Map successfully transferred

coltrane has been setup as an YP slave server without any errors.
Remember to update map ypservers on ellington.
```

This will generate a directory on the slave server called /var/yp/test-domain which contains copies of the NIS master server's maps. Adding these /etc/crontab entries on each slave server will force the slaves to sync their maps with the maps on the master server:

```
20      *       *       *       *       root    /usr/libexec/ypxfr passwd.byname
```

```
21      *       *       *       *       root    /usr/libexec/ypxfr passwd.byuid
```

These entries are not mandatory because the master server automatically attempts to push any map changes to its slaves. However, since clients may depend upon the slave server to provide correct password information, it is recommended to force frequent password map updates. This is especially important on busy networks where map updates might not always complete.

To finish the configuration, run `/etc/netstart` on the slave server in order to start the NIS services.

### 32.4.6. Setting Up an NIS Client

An NIS client binds to an NIS server using ypbind(8). This daemon broadcasts RPC requests on the local network. These requests specify the domain name configured on the client. If an NIS server in the same domain receives one of the broadcasts, it will respond to ypbind, which will record the server's address. If there are several servers available, the client will use the address of the first server to respond and will direct all of its NIS requests to that server. The client will automatically ping the server on a regular basis to make sure it is still available. If it fails to receive a reply within a reasonable amount of time, ypbind will mark the domain as unbound and begin broadcasting again in the hopes of locating another server.

To configure a FreeBSD machine to be an NIS client:

1. Edit /etc/rc.conf and add the following lines in order to set the NIS domain name and start ypbind(8) during network startup:

   ```
   nisdomainname="test-domain"
   nis_client_enable="YES"
   ```

2. To import all possible password entries from the NIS server, use `vipw` to remove all user accounts except one from /etc/master.passwd. When removing the accounts, keep in mind that at least one local account should remain and this account should be a member of `wheel`. If there is a problem with NIS, this local account can be used to log in remotely, become the superuser, and fix the problem. Before saving the edits, add the following line to the end of the file:

   ```
   +:::::::::
   ```

   This line configures the client to provide anyone with a valid account in the NIS server's password maps an account on the client. There are many ways to configure the NIS client by modifying this line. One method is described in Using Netgroups. For more detailed reading, refer to the book `Managing NFS and NIS`, published by O'Reilly Media.

3. To import all possible group entries from the NIS server, add this line to /etc/group:

   ```
   +:*::
   ```

To start the NIS client immediately, execute the following commands as the superuser:

```
# /etc/netstart
# service ypbind start
```

After completing these steps, running `ypcat passwd` on the client should show the server's passwd map.

### 32.4.7. NIS Security

Since RPC is a broadcast-based service, any system running ypbind within the same domain can retrieve the contents of the NIS maps. To prevent unauthorized transactions, ypserv(8) supports a feature called "securenets" which can be used to restrict access to a given set of hosts. By default, this information is stored in /var/yp/securenets, unless ypserv(8) is started with `-p` and an alternate path. This file contains entries that consist of a network specification and a network mask separated by white space. Lines starting with `"#"` are considered to be comments. A sample securenets might look like this:

```
# allow connections from local host -- mandatory
127.0.0.1     255.255.255.255
# allow connections from any host
# on the 192.168.128.0 network
192.168.128.0 255.255.255.0
# allow connections from any host
# between 10.0.0.0 to 10.0.15.255
# this includes the machines in the testlab
10.0.0.0      255.255.240.0
```

If ypserv(8) receives a request from an address that matches one of these rules, it will process the request normally. If the address fails to match a rule, the request will be ignored and a warning message will be logged. If the securenets does not exist, `ypserv` will allow connections from any host.

TCP Wrapper is an alternate mechanism for providing access control instead of securenets. While either access control mechanism adds some security, they are both vulnerable to "IP spoofing" attacks. All NIS-related traffic should be blocked at the firewall.

Servers using securenets may fail to serve legitimate NIS clients with archaic TCP/IP implementations. Some of these implementations set all host bits to zero when doing broadcasts or fail to observe the subnet mask when calculating the broadcast address. While some of these problems can be fixed by changing the client configuration, other problems may force the retirement of these client systems or the abandonment of securenets.

The use of TCP Wrapper increases the latency of the NIS server. The additional delay may be long enough to cause timeouts in client programs, especially in busy networks with slow NIS servers. If one or more clients suffer from latency, convert those clients into NIS slave servers and force them to bind to themselves.

#### 32.4.7.1. Barring Some Users

In this example, the basie system is a faculty workstation within the NIS domain. The passwd map on the master NIS server contains accounts for both faculty and students. This section demonstrates how to allow faculty logins on this system while refusing student logins.

To prevent specified users from logging on to a system, even if they are present in the NIS database, use vipw to add -username with the correct number of colons towards the end of /etc/master.passwd on the client, where *username* is the username of a user to bar from logging in. The line with the blocked user must be before the + line that allows NIS users. In this example, bill is barred from logging on to basie:

```
basie# cat /etc/master.passwd
root:[password]:0:0::0:0:The super-user:/root:/bin/csh
toor:[password]:0:0::0:0:The other super-user:/root:/bin/sh
daemon:*:1:1::0:0:Owner of many system processes:/root:/usr/sbin/nologin
operator:*:2:5::0:0:System &:/:/usr/sbin/nologin
bin:*:3:7::0:0:Binaries Commands and Source,,,:/:/usr/sbin/nologin
tty:*:4:65533::0:0:Tty Sandbox:/:/usr/sbin/nologin
kmem:*:5:65533::0:0:KMem Sandbox:/:/usr/sbin/nologin
games:*:7:13::0:0:Games pseudo-user:/usr/games:/usr/sbin/nologin
news:*:8:8::0:0:News Subsystem:/:/usr/sbin/nologin
man:*:9:9::0:0:Mister Man Pages:/usr/share/man:/usr/sbin/nologin
bind:*:53:53::0:0:Bind Sandbox:/:/usr/sbin/nologin
uucp:*:66:66::0:0:UUCP pseudo-user:/var/spool/uucppublic:/usr/libexec/uucp/uucico
xten:*:67:67::0:0:X-10 daemon:/usr/local/xten:/usr/sbin/nologin
pop:*:68:6::0:0:Post Office Owner:/nonexistent:/usr/sbin/nologin
nobody:*:65534:65534::0:0:Unprivileged user:/nonexistent:/usr/sbin/nologin
-bill:::::::::
+:::::::::

basie#
```

## 32.4.8. Using Netgroups

Barring specified users from logging on to individual systems becomes unscaleable on larger networks and quickly loses the main benefit of NIS: *centralized* administration.

Netgroups were developed to handle large, complex networks with hundreds of users and machines. Their use is comparable to UNIX® groups, where the main difference is the lack of a numeric ID and the ability to define a netgroup by including both user accounts and other netgroups.

To expand on the example used in this chapter, the NIS domain will be extended to add the users and systems shown in Tables 28.2 and 28.3:

*Table 46. Additional Users*

| User Name(s) | Description |
| --- | --- |
| alpha, beta | IT department employees |
| charlie, delta | IT department apprentices |
| echo, foxtrott, golf, ... | employees |
| able, baker, ... | interns |

*Table 47. Additional Systems*

| Machine Name(s) | Description |
| --- | --- |
| war, death, famine, pollution | Only IT employees are allowed to log onto these servers. |
| pride, greed, envy, wrath, lust, sloth | All members of the IT department are allowed to login onto these servers. |
| one, two, three, four, ... | Ordinary workstations used by employees. |
| trashcan | A very old machine without any critical data. Even interns are allowed to use this system. |

When using netgroups to configure this scenario, each user is assigned to one or more netgroups and logins are then allowed or forbidden for all members of the netgroup. When adding a new machine, login restrictions must be defined for all netgroups. When a new user is added, the account must be added to one or more netgroups. If the NIS setup is planned carefully, only one central configuration file needs modification to grant or deny access to machines.

The first step is the initialization of the NIS `netgroup` map. In FreeBSD, this map is not created by default. On the NIS master server, use an editor to create a map named /var/yp/netgroup.

This example creates four netgroups to represent IT employees, IT apprentices, employees, and interns:

```
IT_EMP  (,alpha,test-domain)    (,beta,test-domain)
IT_APP  (,charlie,test-domain)  (,delta,test-domain)
USERS   (,echo,test-domain)     (,foxtrott,test-domain) \
        (,golf,test-domain)
INTERNS (,able,test-domain)     (,baker,test-domain)
```

Each entry configures a netgroup. The first column in an entry is the name of the netgroup. Each set of parentheses represents either a group of one or more users or the name of another netgroup. When specifying a user, the three comma-delimited fields inside each group represent:

1. The name of the host(s) where the other fields representing the user are valid. If a hostname is not specified, the entry is valid on all hosts.
2. The name of the account that belongs to this netgroup.
3. The NIS domain for the account. Accounts may be imported from other NIS domains into a netgroup.

If a group contains multiple users, separate each user with whitespace. Additionally, each field may contain wildcards. See netgroup(5) for details.

Netgroup names longer than 8 characters should not be used. The names are case sensitive and using capital letters for netgroup names is an easy way to distinguish between user, machine and netgroup names.

Some non-FreeBSD NIS clients cannot handle netgroups containing more than 15 entries. This limit may be circumvented by creating several sub-netgroups with 15 users or fewer and a real netgroup consisting of the sub-netgroups, as seen in this example:

```
BIGGRP1  (,joe1,domain)  (,joe2,domain)  (,joe3,domain) [...]
BIGGRP2  (,joe16,domain)  (,joe17,domain) [...]
BIGGRP3  (,joe31,domain)  (,joe32,domain)
BIGGROUP  BIGGRP1 BIGGRP2 BIGGRP3
```

Repeat this process if more than 225 (15 times 15) users exist within a single netgroup.

To activate and distribute the new NIS map:

```
ellington# cd /var/yp
ellington# make
```

This will generate the three NIS maps netgroup, netgroup.byhost and netgroup.byuser. Use the map key option of ypcat(1) to check if the new NIS maps are available:

```
ellington% ypcat -k netgroup
ellington% ypcat -k netgroup.byhost
ellington% ypcat -k netgroup.byuser
```

The output of the first command should resemble the contents of /var/yp/netgroup. The second command only produces output if host-specific netgroups were created. The third command is used to get the list of netgroups for a user.

To configure a client, use vipw(8) to specify the name of the netgroup. For example, on the server named `war`, replace this line:

```
+:::::::::
```

with

```
+@IT_EMP:::::::::
```

This specifies that only the users defined in the netgroup `IT_EMP` will be imported into this system's password database and only those users are allowed to login to this system.

This configuration also applies to the `~` function of the shell and all routines which convert between user names and numerical user IDs. In other words, `cd ~user` will not work, `ls -l` will show the numerical ID instead of the username, and `find . -user joe -print` will fail with the message `No such user`. To fix this, import all user entries without allowing them to login into the servers. This can be achieved by adding an extra line:

```
+::::::::::/usr/sbin/nologin
```

This line configures the client to import all entries but to replace the shell in those entries with /usr/sbin/nologin.

Make sure that extra line is placed *after* `+@IT_EMP::::::::::`. Otherwise, all user accounts imported from NIS will have /usr/sbin/nologin as their login shell and no one will be able to login to the system.

To configure the less important servers, replace the old `+::::::::::` on the servers with these lines:

```
+@IT_EMP::::::::::
+@IT_APP::::::::::
+::::::::::/usr/sbin/nologin
```

The corresponding lines for the workstations would be:

```
+@IT_EMP::::::::::
+@USERS::::::::::
+::::::::::/usr/sbin/nologin
```

NIS supports the creation of netgroups from other netgroups which can be useful if the policy regarding user access changes. One possibility is the creation of role-based netgroups. For example, one might create a netgroup called `BIGSRV` to define the login restrictions for the important servers, another netgroup called `SMALLSRV` for the less important servers, and a third netgroup called `USERBOX` for the workstations. Each of these netgroups contains the netgroups that are allowed to login onto these machines. The new entries for the NIS `netgroup` map would look like this:

```
BIGSRV    IT_EMP  IT_APP
SMALLSRV  IT_EMP  IT_APP  ITINTERN
USERBOX   IT_EMP  ITINTERN USERS
```

This method of defining login restrictions works reasonably well when it is possible to define groups of machines with identical restrictions. Unfortunately, this is the exception and not the rule. Most of the time, the ability to define login restrictions on a per-machine basis is required.

Machine-specific netgroup definitions are another possibility to deal with the policy changes. In this scenario, the /etc/master.passwd of each system contains two lines starting with "+". The first line adds a netgroup with the accounts allowed to login onto this machine and the second line adds

all other accounts with /usr/sbin/nologin as shell. It is recommended to use the "ALL-CAPS" version of the hostname as the name of the netgroup:

```
+@BOXNAME:::::::::
+:::::::::/usr/sbin/nologin
```

Once this task is completed on all the machines, there is no longer a need to modify the local versions of /etc/master.passwd ever again. All further changes can be handled by modifying the NIS map. Here is an example of a possible `netgroup` map for this scenario:

```
# Define groups of users first
IT_EMP    (,alpha,test-domain)    (,beta,test-domain)
IT_APP    (,charlie,test-domain)  (,delta,test-domain)
DEPT1     (,echo,test-domain)     (,foxtrott,test-domain)
DEPT2     (,golf,test-domain)     (,hotel,test-domain)
DEPT3     (,india,test-domain)    (,juliet,test-domain)
ITINTERN  (,kilo,test-domain)     (,lima,test-domain)
D_INTERNS (,able,test-domain)     (,baker,test-domain)
#
# Now, define some groups based on roles
USERS     DEPT1   DEPT2     DEPT3
BIGSRV    IT_EMP  IT_APP
SMALLSRV  IT_EMP  IT_APP    ITINTERN
USERBOX   IT_EMP  ITINTERN  USERS
#
# And a groups for a special tasks
# Allow echo and golf to access our anti-virus-machine
SECURITY  IT_EMP  (,echo,test-domain)  (,golf,test-domain)
#
# machine-based netgroups
# Our main servers
WAR       BIGSRV
FAMINE    BIGSRV
# User india needs access to this server
POLLUTION  BIGSRV  (,india,test-domain)
#
# This one is really important and needs more access restrictions
DEATH     IT_EMP
#
# The anti-virus-machine mentioned above
ONE       SECURITY
#
# Restrict a machine to a single user
TWO       (,hotel,test-domain)
# [...more groups to follow]
```

It may not always be advisable to use machine-based netgroups. When deploying a couple of dozen or hundreds of systems, role-based netgroups instead of machine-based netgroups may be used to

keep the size of the NIS map within reasonable limits.

### 32.4.9. Password Formats

NIS requires that all hosts within an NIS domain use the same format for encrypting passwords. If users have trouble authenticating on an NIS client, it may be due to a differing password format. In a heterogeneous network, the format must be supported by all operating systems, where DES is the lowest common standard.

To check which format a server or client is using, look at this section of /etc/login.conf:

```
default:\
    :passwd_format=des:\
    [Further entries elided]
```

In this example, the system is using the DES format for password hashing. Other possible values include `blf` for Blowfish, `md5` for MD5, `sha256` and `sha512` for SHA-256 and SHA-512 respectively. For more information and the up to date list of what is available on the system, consult the crypt(3) manpage.

If the format on a host needs to be edited to match the one being used in the NIS domain, the login capability database must be rebuilt after saving the change:

```
# cap_mkdb /etc/login.conf
```



The format of passwords for existing user accounts will not be updated until each user changes their password *after* the login capability database is rebuilt.

## 32.5. Lightweight Directory Access Protocol (LDAP)

The Lightweight Directory Access Protocol (LDAP) is an application layer protocol used to access, modify, and authenticate objects using a distributed directory information service. Think of it as a phone or record book which stores several levels of hierarchical, homogeneous information. It is used in Active Directory and OpenLDAP networks and allows users to access to several levels of internal information utilizing a single account. For example, email authentication, pulling employee contact information, and internal website authentication might all make use of a single user account in the LDAP server's record base.

This section provides a quick start guide for configuring an LDAP server on a FreeBSD system. It assumes that the administrator already has a design plan which includes the type of information to store, what that information will be used for, which users should have access to that information, and how to secure this information from unauthorized access.

### 32.5.1. LDAP Terminology and Structure

LDAP uses several terms which should be understood before starting the configuration. All

directory entries consist of a group of *attributes*. Each of these attribute sets contains a unique identifier known as a *Distinguished Name* (DN) which is normally built from several other attributes such as the common or *Relative Distinguished Name* (RDN). Similar to how directories have absolute and relative paths, consider a DN as an absolute path and the RDN as the relative path.

An example LDAP entry looks like the following. This example searches for the entry for the specified user account (`uid`), organizational unit (`ou`), and organization (`o`):

```
% ldapsearch -xb "uid=trhodes,ou=users,o=example.com"
# extended LDIF
#
# LDAPv3
# base <uid=trhodes,ou=users,o=example.com> with scope subtree
# filter: (objectclass=*)
# requesting: ALL
#

# trhodes, users, example.com
dn: uid=trhodes,ou=users,o=example.com
mail: trhodes@example.com
cn: Tom Rhodes
uid: trhodes
telephoneNumber: (123) 456-7890

# search result
search: 2
result: 0 Success

# numResponses: 2
# numEntries: 1
```

This example entry shows the values for the `dn`, `mail`, `cn`, `uid`, and `telephoneNumber` attributes. The cn attribute is the RDN.

More information about LDAP and its terminology can be found at http://www.openldap.org/doc/admin24/intro.html.

### 32.5.2. Configuring an LDAP Server

FreeBSD does not provide a built-in LDAP server. Begin the configuration by installing net/openldap-server package or port:

```
# pkg install openldap-server
```

There is a large set of default options enabled in the package. Review them by running `pkg info openldap-server`. If they are not sufficient (for example if SQL support is needed), please consider recompiling the port using the appropriate framework.

The installation creates the directory /var/db/openldap-data to hold the data. The directory to store the certificates must be created:

```
# mkdir /usr/local/etc/openldap/private
```

The next phase is to configure the Certificate Authority. The following commands must be executed from /usr/local/etc/openldap/private. This is important as the file permissions need to be restrictive and users should not have access to these files. More detailed information about certificates and their parameters can be found in OpenSSL. To create the Certificate Authority, start with this command and follow the prompts:

```
# openssl req -days 365 -nodes -new -x509 -keyout ca.key -out ../ca.crt
```

The entries for the prompts may be generic *except* for the `Common Name`. This entry must be *different* than the system hostname. If this will be a self signed certificate, prefix the hostname with `CA` for Certificate Authority.

The next task is to create a certificate signing request and a private key. Input this command and follow the prompts:

```
# openssl req -days 365 -nodes -new -keyout server.key -out server.csr
```

During the certificate generation process, be sure to correctly set the `Common Name` attribute. The Certificate Signing Request must be signed with the Certificate Authority in order to be used as a valid certificate:

```
# openssl x509 -req -days 365 -in server.csr -out ../server.crt -CA ../ca.crt -CAkey
ca.key -CAcreateserial
```

The final part of the certificate generation process is to generate and sign the client certificates:

```
# openssl req -days 365 -nodes -new -keyout client.key -out client.csr
# openssl x509 -req -days 3650 -in client.csr -out ../client.crt -CA ../ca.crt -CAkey
ca.key
```

Remember to use the same `Common Name` attribute when prompted. When finished, ensure that a total of eight (8) new files have been generated through the proceeding commands.

The daemon running the OpenLDAP server is slapd. Its configuration is performed through slapd.ldif: the old slapd.conf has been deprecated by OpenLDAP.

Configuration examples for slapd.ldif are available and can also be found in /usr/local/etc/openldap/slapd.ldif.sample. Options are documented in slapd-config(5). Each section of slapd.ldif, like all the other LDAP attribute sets, is uniquely identified through a DN. Be sure that

no blank lines are left between the `dn:` statement and the desired end of the section. In the following example, TLS will be used to implement a secure channel. The first section represents the global configuration:

```
#
# See slapd-config(5) for details on configuration options.
# This file should NOT be world readable.
#
dn: cn=config
objectClass: olcGlobal
cn: config
#
#
# Define global ACLs to disable default read access.
#
olcArgsFile: /var/run/openldap/slapd.args
olcPidFile: /var/run/openldap/slapd.pid
olcTLSCertificateFile: /usr/local/etc/openldap/server.crt
olcTLSCertificateKeyFile: /usr/local/etc/openldap/private/server.key
olcTLSCACertificateFile: /usr/local/etc/openldap/ca.crt
#olcTLSCipherSuite: HIGH
olcTLSProtocolMin: 3.1
olcTLSVerifyClient: never
```

The Certificate Authority, server certificate and server private key files must be specified here. It is recommended to let the clients choose the security cipher and omit option `olcTLSCipherSuite` (incompatible with TLS clients other than openssl). Option `olcTLSProtocolMin` lets the server require a minimum security level: it is recommended. While verification is mandatory for the server, it is not for the client: `olcTLSVerifyClient: never`.

The second section is about the backend modules and can be configured as follows:

```
#
# Load dynamic backend modules:
#
dn: cn=module,cn=config
objectClass: olcModuleList
cn: module
olcModulepath:  /usr/local/libexec/openldap
olcModuleload:  back_mdb.la
#olcModuleload: back_bdb.la
#olcModuleload: back_hdb.la
#olcModuleload: back_ldap.la
#olcModuleload: back_passwd.la
#olcModuleload: back_shell.la
```

The third section is devoted to load the needed `ldif` schemas to be used by the databases: they are essential.

```
dn: cn=schema,cn=config
objectClass: olcSchemaConfig
cn: schema

include: file:///usr/local/etc/openldap/schema/core.ldif
include: file:///usr/local/etc/openldap/schema/cosine.ldif
include: file:///usr/local/etc/openldap/schema/inetorgperson.ldif
include: file:///usr/local/etc/openldap/schema/nis.ldif
```

Next, the frontend configuration section:

```
# Frontend settings
#
dn: olcDatabase={-1}frontend,cn=config
objectClass: olcDatabaseConfig
objectClass: olcFrontendConfig
olcDatabase: {-1}frontend
olcAccess: to * by * read
#
# Sample global access control policy:
#   Root DSE: allow anyone to read it
#   Subschema (sub)entry DSE: allow anyone to read it
#   Other DSEs:
#       Allow self write access
#       Allow authenticated users read access
#       Allow anonymous users to authenticate
#
#olcAccess: to dn.base="" by * read
#olcAccess: to dn.base="cn=Subschema" by * read
#olcAccess: to *
#   by self write
#   by users read
#   by anonymous auth
#
# if no access controls are present, the default policy
# allows anyone and everyone to read anything but restricts
# updates to rootdn.  (e.g., "access to * by * read")
#
# rootdn can always read and write EVERYTHING!
#
olcPasswordHash: {SSHA}
# {SSHA} is already the default for olcPasswordHash
```

Another section is devoted to the *configuration backend*, the only way to later access the OpenLDAP server configuration is as a global super-user.

```
dn: olcDatabase={0}config,cn=config
objectClass: olcDatabaseConfig
```

```
olcDatabase: {0}config
olcAccess: to * by * none
olcRootPW: {SSHA}iae+lrQZILpiUdf16Z9KmDmSwT77Dj4U
```

The default administrator username is `cn=config`. Type slappasswd in a shell, choose a password and use its hash in `olcRootPW`. If this option is not specified now, before slapd.ldif is imported, no one will be later able to modify the *global configuration* section.

The last section is about the database backend:

```
#######################################################################
# LMDB database definitions
#######################################################################
#
dn: olcDatabase=mdb,cn=config
objectClass: olcDatabaseConfig
objectClass: olcMdbConfig
olcDatabase: mdb
olcDbMaxSize: 1073741824
olcSuffix: dc=domain,dc=example
olcRootDN: cn=mdbadmin,dc=domain,dc=example
# Cleartext passwords, especially for the rootdn, should
# be avoided.  See slappasswd(8) and slapd-config(5) for details.
# Use of strong authentication encouraged.
olcRootPW: {SSHA}X2wHvIWDk6G76CQyCMS1vDCvtICWgn0+
# The database directory MUST exist prior to running slapd AND
# should only be accessible by the slapd and slap tools.
# Mode 700 recommended.
olcDbDirectory: /var/db/openldap-data
# Indices to maintain
olcDbIndex: objectClass eq
```

This database hosts the *actual contents* of the LDAP directory. Types other than `mdb` are available. Its super-user, not to be confused with the global one, is configured here: a (possibly custom) username in `olcRootDN` and the password hash in `olcRootPW`; slappasswd can be used as before.

This repository contains four examples of slapd.ldif. To convert an existing slapd.conf into slapd.ldif, refer to this page (please note that this may introduce some unuseful options).

When the configuration is completed, slapd.ldif must be placed in an empty directory. It is recommended to create it as:

```
# mkdir /usr/local/etc/openldap/slapd.d/
```

Import the configuration database:

```
# /usr/local/sbin/slapadd -n0 -F /usr/local/etc/openldap/slapd.d/ -l
```

```
/usr/local/etc/openldap/slapd.ldif
```

Start the slapd daemon:

```
# /usr/local/libexec/slapd -F /usr/local/etc/openldap/slapd.d/
```

Option `-d` can be used for debugging, as specified in slapd(8). To verify that the server is running and working:

```
# ldapsearch -x -b '' -s base '(objectclass=*)' namingContexts
# extended LDIF
#
# LDAPv3
# base <> with scope baseObject
# filter: (objectclass=*)
# requesting: namingContexts
#

#
dn:
namingContexts: dc=domain,dc=example

# search result
search: 2
result: 0 Success

# numResponses: 2
# numEntries: 1
```

The server must still be trusted. If that has never been done before, follow these instructions. Install the OpenSSL package or port:

```
# pkg install openssl
```

From the directory where ca.crt is stored (in this example, /usr/local/etc/openldap), run:

```
# c_rehash .
```

Both the CA and the server certificate are now correctly recognized in their respective roles. To verify this, run this command from the server.crt directory:

```
# openssl verify -verbose -CApath . server.crt
```

If slapd was running, restart it. As stated in /usr/local/etc/rc.d/slapd, to properly run slapd at boot

the following lines must be added to /etc/rc.conf:

```
slapd_enable="YES"
slapd_flags='-h "ldapi://%2fvar%2frun%2fopenldap%2fldapi/ ldap://0.0.0.0/"'
slapd_sockets="/var/run/openldap/ldapi"
slapd_cn_config="YES"
```

slapd does not provide debugging at boot. Check /var/log/debug.log, dmesg -a and /var/log/messages for this purpose.

The following example adds the group `team` and the user `john` to the `domain.example` LDAP database, which is still empty. First, create the file domain.ldif:

```
# cat domain.ldif
dn: dc=domain,dc=example
objectClass: dcObject
objectClass: organization
o: domain.example
dc: domain

dn: ou=groups,dc=domain,dc=example
objectClass: top
objectClass: organizationalunit
ou: groups

dn: ou=users,dc=domain,dc=example
objectClass: top
objectClass: organizationalunit
ou: users

dn: cn=team,ou=groups,dc=domain,dc=example
objectClass: top
objectClass: posixGroup
cn: team
gidNumber: 10001

dn: uid=john,ou=users,dc=domain,dc=example
objectClass: top
objectClass: account
objectClass: posixAccount
objectClass: shadowAccount
cn: John McUser
uid: john
uidNumber: 10001
gidNumber: 10001
homeDirectory: /home/john/
loginShell: /usr/bin/bash
userPassword: secret
```

See the OpenLDAP documentation for more details. Use slappasswd to replace the plain text password `secret` with a hash in `userPassword`. The path specified as `loginShell` must exist in all the systems where `john` is allowed to login. Finally, use the `mdb` administrator to modify the database:

```
# ldapadd -W -D "cn=mdbadmin,dc=domain,dc=example" -f domain.ldif
```

Modifications to the *global configuration* section can only be performed by the global super-user. For example, assume that the option `olcTLSCipherSuite: HIGH:MEDIUM:SSLv3` was initially specified and must now be deleted. First, create a file that contains the following:

```
# cat global_mod
dn: cn=config
changetype: modify
delete: olcTLSCipherSuite
```

Then, apply the modifications:

```
# ldapmodify -f global_mod -x -D "cn=config" -W
```

When asked, provide the password chosen in the *configuration backend* section. The username is not required: here, `cn=config` represents the DN of the database section to be modified. Alternatively, use `ldapmodify` to delete a single line of the database, `ldapdelete` to delete a whole entry.

If something goes wrong, or if the global super-user cannot access the configuration backend, it is possible to delete and re-write the whole configuration:

```
# rm -rf /usr/local/etc/openldap/slapd.d/
```

slapd.ldif can then be edited and imported again. Please, follow this procedure only when no other solution is available.

This is the configuration of the server only. The same machine can also host an LDAP client, with its own separate configuration.

## 32.6. Dynamic Host Configuration Protocol (DHCP)

The Dynamic Host Configuration Protocol (DHCP) allows a system to connect to a network in order to be assigned the necessary addressing information for communication on that network. FreeBSD includes the OpenBSD version of `dhclient` which is used by the client to obtain the addressing information. FreeBSD does not install a DHCP server, but several servers are available in the FreeBSD Ports Collection. The DHCP protocol is fully described in RFC 2131. Informational resources are also available at isc.org/downloads/dhcp/.

This section describes how to use the built-in DHCP client. It then describes how to install and

configure a DHCP server.



In FreeBSD, the bpf(4) device is needed by both the DHCP server and DHCP client. This device is included in the GENERIC kernel that is installed with FreeBSD. Users who prefer to create a custom kernel need to keep this device if DHCP is used.

It should be noted that bpf also allows privileged users to run network packet sniffers on that system.

### 32.6.1. Configuring a DHCP Client

DHCP client support is included in the FreeBSD installer, making it easy to configure a newly installed system to automatically receive its networking addressing information from an existing DHCP server. Refer to Accounts, Time Zone, Services and Hardening for examples of network configuration.

When `dhclient` is executed on the client machine, it begins broadcasting requests for configuration information. By default, these requests use UDP port 68. The server replies on UDP port 67, giving the client an IP address and other relevant network information such as a subnet mask, default gateway, and DNS server addresses. This information is in the form of a DHCP "lease" and is valid for a configurable time. This allows stale IP addresses for clients no longer connected to the network to automatically be reused. DHCP clients can obtain a great deal of information from the server. An exhaustive list may be found in dhcp-options(5).

By default, when a FreeBSD system boots, its DHCP client runs in the background, or *asynchronously*. Other startup scripts continue to run while the DHCP process completes, which speeds up system startup.

Background DHCP works well when the DHCP server responds quickly to the client's requests. However, DHCP may take a long time to complete on some systems. If network services attempt to run before DHCP has assigned the network addressing information, they will fail. Using DHCP in *synchronous* mode prevents this problem as it pauses startup until the DHCP configuration has completed.

This line in /etc/rc.conf is used to configure background or asynchronous mode:

```
ifconfig_fxp0="DHCP"
```

This line may already exist if the system was configured to use DHCP during installation. Replace the *fxp0* shown in these examples with the name of the interface to be dynamically configured, as described in "Setting Up Network Interface Cards".

To instead configure the system to use synchronous mode, and to pause during startup while DHCP completes, use "SYNCDHCP":

```
ifconfig_fxp0="SYNCDHCP"
```

Additional client options are available. Search for `dhclient` in rc.conf(5) for details.

The DHCP client uses the following files:

- /etc/dhclient.conf

  The configuration file used by `dhclient`. Typically, this file contains only comments as the defaults are suitable for most clients. This configuration file is described in dhclient.conf(5).

- /sbin/dhclient

  More information about the command itself can be found in dhclient(8).

- /sbin/dhclient-script

  The FreeBSD-specific DHCP client configuration script. It is described in dhclient-script(8), but should not need any user modification to function properly.

- /var/db/dhclient.leases.interface

  The DHCP client keeps a database of valid leases in this file, which is written as a log and is described in dhclient.leases(5).

### 32.6.2. Installing and Configuring a DHCP Server

This section demonstrates how to configure a FreeBSD system to act as a DHCP server using the Internet Systems Consortium (ISC) implementation of the DHCP server. This implementation and its documentation can be installed using the net/isc-dhcp44-server package or port.

The installation of net/isc-dhcp44-server installs a sample configuration file. Copy /usr/local/etc/dhcpd.conf.example to /usr/local/etc/dhcpd.conf and make any edits to this new file.

The configuration file is comprised of declarations for subnets and hosts which define the information that is provided to DHCP clients. For example, these lines configure the following:

```
option domain-name "example.org";①
option domain-name-servers ns1.example.org;②
option subnet-mask 255.255.255.0;③

default-lease-time 600;④
max-lease-time 72400;⑤
ddns-update-style none;⑥

subnet 10.254.239.0 netmask 255.255.255.224 {
  range 10.254.239.10 10.254.239.20;⑦
  option routers rtr-239-0-1.example.org, rtr-239-0-2.example.org;⑧
}

host fantasia {
  hardware ethernet 08:00:07:26:c0:a5;⑨
  fixed-address fantasia.fugue.com;⑩
```

```
}
```

① This option specifies the default search domain that will be provided to clients. Refer to resolv.conf(5) for more information.

② This option specifies a comma separated list of DNS servers that the client should use. They can be listed by their Fully Qualified Domain Names (FQDN), as seen in the example, or by their IP addresses.

③ The subnet mask that will be provided to clients.

④ The default lease expiry time in seconds. A client can be configured to override this value.

⑤ The maximum allowed length of time, in seconds, for a lease. Should a client request a longer lease, a lease will still be issued, but it will only be valid for `max-lease-time`.

⑥ The default of `none` disables dynamic DNS updates. Changing this to `interim` configures the DHCP server to update a DNS server whenever it hands out a lease so that the DNS server knows which IP addresses are associated with which computers in the network. Do not change the default setting unless the DNS server has been configured to support dynamic DNS.

⑦ This line creates a pool of available IP addresses which are reserved for allocation to DHCP clients. The range of addresses must be valid for the network or subnet specified in the previous line.

⑧ Declares the default gateway that is valid for the network or subnet specified before the opening `{` bracket.

⑨ Specifies the hardware MAC address of a client so that the DHCP server can recognize the client when it makes a request.

⑩ Specifies that this host should always be given the same IP address. Using the hostname is correct, since the DHCP server will resolve the hostname before returning the lease information.

This configuration file supports many more options. Refer to dhcpd.conf(5), installed with the server, for details and examples.

Once the configuration of dhcpd.conf is complete, enable the DHCP server in /etc/rc.conf:

```
dhcpd_enable="YES"
dhcpd_ifaces="dc0"
```

Replace the `dc0` with the interface (or interfaces, separated by whitespace) that the DHCP server should listen on for DHCP client requests.

Start the server by issuing the following command:

```
# service isc-dhcpd start
```

Any future changes to the configuration of the server will require the dhcpd service to be stopped and then started using service(8).

The DHCP server uses the following files. Note that the manual pages are installed with the server software.

- /usr/local/sbin/dhcpd

  More information about the dhcpd server can be found in dhcpd(8).

- /usr/local/etc/dhcpd.conf

  The server configuration file needs to contain all the information that should be provided to clients, along with information regarding the operation of the server. This configuration file is described in dhcpd.conf(5).

- /var/db/dhcpd.leases

  The DHCP server keeps a database of leases it has issued in this file, which is written as a log. Refer to dhcpd.leases(5), which gives a slightly longer description.

- /usr/local/sbin/dhcrelay

  This daemon is used in advanced environments where one DHCP server forwards a request from a client to another DHCP server on a separate network. If this functionality is required, install the net/isc-dhcp44-relay package or port. The installation includes dhcrelay(8) which provides more detail.

## 32.7. Domain Name System (DNS)

Domain Name System (DNS) is the protocol through which domain names are mapped to IP addresses, and vice versa. DNS is coordinated across the Internet through a somewhat complex system of authoritative root, Top Level Domain (TLD), and other smaller-scale name servers, which host and cache individual domain information. It is not necessary to run a name server to perform DNS lookups on a system.

The following table describes some of the terms associated with DNS:

*Table 48. DNS Terminology*

| Term | Definition |
|---|---|
| Forward DNS | Mapping of hostnames to IP addresses. |
| Origin | Refers to the domain covered in a particular zone file. |
| Resolver | A system process through which a machine queries a name server for zone information. |
| Reverse DNS | Mapping of IP addresses to hostnames. |
| Root zone | The beginning of the Internet zone hierarchy. All zones fall under the root zone, similar to how all files in a file system fall under the root directory. |

| Term | Definition |
| --- | --- |
| Zone | An individual domain, subdomain, or portion of the DNS administered by the same authority. |

Examples of zones:

- `.` is how the root zone is usually referred to in documentation.
- `org.` is a Top Level Domain (TLD) under the root zone.
- `example.org.` is a zone under the `org.`TLD.
- `1.168.192.in-addr.arpa` is a zone referencing all IP addresses which fall under the `192.168.1.*`IP address space.

As one can see, the more specific part of a hostname appears to its left. For example, `example.org.` is more specific than `org.`, as `org.` is more specific than the root zone. The layout of each part of a hostname is much like a file system: the /dev directory falls within the root, and so on.

### 32.7.1. Reasons to Run a Name Server

Name servers generally come in two forms: authoritative name servers, and caching (also known as resolving) name servers.

An authoritative name server is needed when:

- One wants to serve DNS information to the world, replying authoritatively to queries.
- A domain, such as `example.org`, is registered and IP addresses need to be assigned to hostnames under it.
- An IP address block requires reverse DNS entries (IP to hostname).
- A backup or second name server, called a slave, will reply to queries.

A caching name server is needed when:

- A local DNS server may cache and respond more quickly than querying an outside name server.

When one queries for `www.FreeBSD.org`, the resolver usually queries the uplink ISP's name server, and retrieves the reply. With a local, caching DNS server, the query only has to be made once to the outside world by the caching DNS server. Additional queries will not have to go outside the local network, since the information is cached locally.

### 32.7.2. DNS Server Configuration

Unbound is provided in the FreeBSD base system. By default, it will provide DNS resolution to the local machine only. While the base system package can be configured to provide resolution services beyond the local machine, it is recommended that such requirements be addressed by installing Unbound from the FreeBSD Ports Collection.

To enable Unbound, add the following to /etc/rc.conf:

```
local_unbound_enable="YES"
```

Any existing nameservers in /etc/resolv.conf will be configured as forwarders in the new Unbound configuration.



If any of the listed nameservers do not support DNSSEC, local DNS resolution will fail. Be sure to test each nameserver and remove any that fail the test. The following command will show the trust tree or a failure for a nameserver running on 192.168.1.1:

```
% drill -S FreeBSD.org @192.168.1.1
```

Once each nameserver is confirmed to support DNSSEC, start Unbound:

```
# service local_unbound onestart
```

This will take care of updating /etc/resolv.conf so that queries for DNSSEC secured domains will now work. For example, run the following to validate the FreeBSD.org DNSSEC trust tree:

```
% drill -S FreeBSD.org
;; Number of trusted keys: 1
;; Chasing: freebsd.org. A

DNSSEC Trust tree:
freebsd.org. (A)
|---freebsd.org. (DNSKEY keytag: 36786 alg: 8 flags: 256)
    |---freebsd.org. (DNSKEY keytag: 32659 alg: 8 flags: 257)
    |---freebsd.org. (DS keytag: 32659 digest type: 2)
        |---org. (DNSKEY keytag: 49587 alg: 7 flags: 256)
            |---org. (DNSKEY keytag: 9795 alg: 7 flags: 257)
            |---org. (DNSKEY keytag: 21366 alg: 7 flags: 257)
            |---org. (DS keytag: 21366 digest type: 1)
            |   |---. (DNSKEY keytag: 40926 alg: 8 flags: 256)
            |       |---. (DNSKEY keytag: 19036 alg: 8 flags: 257)
            |---org. (DS keytag: 21366 digest type: 2)
                |---. (DNSKEY keytag: 40926 alg: 8 flags: 256)
                    |---. (DNSKEY keytag: 19036 alg: 8 flags: 257)
;; Chase successful
```

### 32.7.3. Authoritative Name Server Configuration

FreeBSD does not provide authoritative name server software in the base system. Users are encouraged to install third party applications, like dns/nsd or dns/bind918 package or port.

# 32.8. Zero-configuration Networking (mDNS/DNS-SD)

Zero-configuration networking (sometimes referred to as *Zeroconf*) is a set of technologies, which simplify network configuration. The main parts of Zeroconf are:

- Link-Local Addressing providing automatic assignment of numeric network addresses.
- Multicast DNS (*mDNS*) providing automatic distribution and resolution of hostnames.
- DNS-Based Service Discovery (*DNS-SD*) providing automatic discovery of service instances.

## 32.8.1. Configuring and Starting Avahi

One of the popular implementations of zeroconf is Avahi. Avahi can be installed and configured with the following commands:

```
# pkg install avahi-app nss_mdns
# grep -q '^hosts:.*\<mdns\>' /etc/nsswitch.conf || sed -i "" 's/^hosts: .*/& mdns/'
/etc/nsswitch.conf
# service dbus enable
# service avahi-daemon enable
# service dbus start
# service avahi-daemon start
```

# 32.9. Apache HTTP Server

The open source Apache HTTP Server is the most widely used web server. FreeBSD does not install this web server by default, but it can be installed from the www/apache24 package or port.

This section summarizes how to configure and start version 2.*x* of the Apache HTTP Server on FreeBSD. For more detailed information about Apache 2.X and its configuration directives, refer to httpd.apache.org.

## 32.9.1. Configuring and Starting Apache

In FreeBSD, the main Apache HTTP Server configuration file is installed as /usr/local/etc/apache2x/httpd.conf, where *x* represents the version number. This ASCII text file begins comment lines with a `#`. The most frequently modified directives are:

`ServerRoot "/usr/local"`

Specifies the default directory hierarchy for the Apache installation. Binaries are stored in the bin and sbin subdirectories of the server root and configuration files are stored in the etc/apache2x subdirectory.

`ServerAdmin you@example.com`

Change this to the email address to receive problems with the server. This address also appears on some server-generated pages, such as error documents.

`ServerName www.example.com:80`

Allows an administrator to set a hostname which is sent back to clients for the server. For example, `www` can be used instead of the actual hostname. If the system does not have a registered DNS name, enter its IP address instead. If the server will listen on an alternate report, change `80` to the alternate port number.

`DocumentRoot "/usr/local/www/apache2_x_/data"`

The directory where documents will be served from. By default, all requests are taken from this directory, but symbolic links and aliases may be used to point to other locations.

It is always a good idea to make a backup copy of the default Apache configuration file before making changes. When the configuration of Apache is complete, save the file and verify the configuration using `apachectl`. Running `apachectl configtest` should return `Syntax OK`.

To launch Apache at system startup, add the following line to /etc/rc.conf:

```
apache24_enable="YES"
```

If Apache should be started with non-default options, the following line may be added to /etc/rc.conf to specify the needed flags:

```
apache24_flags=""
```

If apachectl does not report configuration errors, start `httpd` now:

```
# service apache24 start
```

The `httpd` service can be tested by entering `http://localhost` in a web browser, replacing *localhost* with the fully-qualified domain name of the machine running `httpd`. The default web page that is displayed is /usr/local/www/apache24/data/index.html.

The Apache configuration can be tested for errors after making subsequent configuration changes while `httpd` is running using the following command:

```
# service apache24 configtest
```



It is important to note that `configtest` is not an rc(8) standard, and should not be expected to work for all startup scripts.

### 32.9.2. Virtual Hosting

Virtual hosting allows multiple websites to run on one Apache server. The virtual hosts can be *IP-based* or *name-based*. IP-based virtual hosting uses a different IP address for each website. Name-based virtual hosting uses the clients HTTP/1.1 headers to figure out the hostname, which allows

the websites to share the same IP address.

To setup Apache to use name-based virtual hosting, add a `VirtualHost` block for each website. For example, for the webserver named `www.domain.tld` with a virtual domain of `www.someotherdomain.tld`, add the following entries to httpd.conf:

```
<VirtualHost *>
    ServerName www.domain.tld
    DocumentRoot /www/domain.tld
</VirtualHost>

<VirtualHost *>
    ServerName www.someotherdomain.tld
    DocumentRoot /www/someotherdomain.tld
</VirtualHost>
```

For each virtual host, replace the values for `ServerName` and `DocumentRoot` with the values to be used.

For more information about setting up virtual hosts, consult the official Apache documentation at: http://httpd.apache.org/docs/vhosts/.

### 32.9.3. Apache Modules

Apache uses modules to augment the functionality provided by the basic server. Refer to http://httpd.apache.org/docs/current/mod/ for a complete listing of and the configuration details for the available modules.

In FreeBSD, some modules can be compiled with the www/apache24 port. Type `make config` within /usr/ports/www/apache24 to see which modules are available and which are enabled by default. If the module is not compiled with the port, the FreeBSD Ports Collection provides an easy way to install many modules. This section describes three of the most commonly used modules.

#### 32.9.3.1. SSL support

At one point, support for SSL inside of Apache required a secondary module called mod_ssl. This is no longer the case and the default install of Apache comes with SSL built into the web server. An example of how to enable support for SSL websites is available in the installed file, httpd-ssl.conf inside of the /usr/local/etc/apache24/extra directory Inside this directory is also a sample file called named ssl.conf-sample. It is recommended that both files be evaluated to properly set up secure websites in the Apache web server.

After the configuration of SSL is complete, the following line must be uncommented in the main http.conf to activate the changes on the next restart or reload of Apache:

```
#Include etc/apache24/extra/httpd-ssl.conf
```



SSL version two and version three have known vulnerability issues. It is highly recommended TLS version 1.2 and 1.3 be enabled in place of the older SSL options.

> This can be accomplished by setting the following options in the ssl.conf:

```
SSLProtocol all -SSLv3 -SSLv2 +TLSv1.2 +TLSv1.3
SSLProxyProtocol all -SSLv2 -SSLv3 -TLSv1 -TLSv1.1
```

To complete the configuration of SSL in the web server, uncomment the following line to ensure that the configuration will be pulled into Apache during restart or reload:

```
# Secure (SSL/TLS) connections
Include etc/apache24/extra/httpd-ssl.conf
```

The following lines must also be uncommented in the httpd.conf to fully support SSL in Apache:

```
LoadModule authn_socache_module libexec/apache24/mod_authn_socache.so
LoadModule socache_shmcb_module libexec/apache24/mod_socache_shmcb.so
LoadModule ssl_module libexec/apache24/mod_ssl.so
```

The next step is to work with a certificate authority to have the appropriate certificates installed on the system. This will set up a chain of trust for the site and prevent any warnings of self-signed certificates.

#### 32.9.3.2. mod_perl

The mod_perl module makes it possible to write Apache modules in Perl. In addition, the persistent interpreter embedded in the server avoids the overhead of starting an external interpreter and the penalty of Perl start-up time.

The mod_perl can be installed using the www/mod_perl2 package or port. Documentation for using this module can be found at http://perl.apache.org/docs/2.0/index.html.

#### 32.9.3.3. mod_php

*PHP: Hypertext Preprocessor* (PHP) is a general-purpose scripting language that is especially suited for web development. Capable of being embedded into HTML, its syntax draws upon C, Java™, and Perl with the intention of allowing web developers to write dynamically generated webpages quickly.

Support for PHP for Apache and any other feature written in the language, can be added by installing the appropriate port.

For all supported versions, search the package database using `pkg`:

```
# pkg search php
```

A list will be displayed including the versions and additional features they provide. The components are completely modular, meaning features are enabled by installing the appropriate

port. To install PHP version 7.4 for Apache, issue the following command:

```
# pkg install mod_php74
```

If any dependency packages need to be installed, they will be installed as well.

By default, PHP will not be enabled. The following lines will need to be added to the Apache configuration file located in /usr/local/etc/apache24 to make it active:

```
<FilesMatch "\.php$">
    SetHandler application/x-httpd-php
</FilesMatch>
<FilesMatch "\.phps$">
    SetHandler application/x-httpd-php-source
</FilesMatch>
```

In addition, the `DirectoryIndex` in the configuration file will also need to be updated and Apache will either need to be restarted or reloaded for the changes to take effect.

Support for many of the PHP features may also be installed by using `pkg`. For example, to install support for XML or SSL, install their respective ports:

```
# pkg install php74-xml php74-openssl
```

As before, the Apache configuration will need to be reloaded for the changes to take effect, even in cases where it was just a module install.

To perform a graceful restart to reload the configuration, issue the following command:

```
# apachectl graceful
```

Once the install is complete, there are two methods of obtaining the installed PHP support modules and the environmental information of the build. The first is to install the full PHP binary and running the command to gain the information:

```
# pkg install php74
```

```
# php -i | less
```

It is necessary to pass the output to a pager, such as the `more` or `less` to easier digest the amount of output.

Finally, to make any changes to the global configuration of PHP there is a well documented file installed into /usr/local/etc/php.ini. At the time of install, this file will not exist because there are

two versions to choose from, one is php.ini-development and the other is php.ini-production. These are starting points to assist administrators in their deployment.

#### 32.9.3.4. HTTP2 Support

Apache support for the HTTP2 protocol is included by default when installing the port with `pkg`. The new version of HTTP includes many improvements over the previous version, including utilizing a single connection to a website, reducing overall roundtrips of TCP connections. Also, packet header data is compressed and HTTP2 requires encryption by default.

When Apache is configured to only use HTTP2, web browsers will require secure, encrypted HTTPS connections. When Apache is configured to use both versions, HTTP1.1 will be considered a fall back option if any issues arise during the connection.

While this change does require administrators to make changes, they are positive and equate to a more secure Internet for everyone. The changes are only required for sites not currently implementing SSL and TLS.



This configuration depends on the previous sections, including TLS support. It is recommended those instructions be followed before continuing with this configuration.

Start the process by enabling the http2 module by uncommenting the line in /usr/local/etc/apache24/httpd.conf and replace the mpm_prefork module with mpm_event as the former does not support HTTP2.

```
LoadModule http2_module libexec/apache24/mod_http2.so
LoadModule mpm_event_module libexec/apache24/mod_mpm_event.so
```



There is a separate mod_http2 port that is available. It exists to deliver security and bug fixes quicker than the module installed with the bundled apache24 port. It is not required for HTTP2 support but is available. When installed, the mod_h2.so should be used in place of mod_http2.so in the Apache configuration.

There are two methods to implement HTTP2 in Apache; one way is globally for all sites and each VirtualHost running on the system. To enable HTTP2 globally, add the following line under the ServerName directive:

```
Protocols h2 http/1.1
```



To enable HTTP2 over plaintext, use h2h2chttp/1.1 in the httpd.conf.

Having the h2c here will allow plaintext HTTP2 data to pass on the system but is not recommended. In addition, using the http/1.1 here will allow fallback to the HTTP1.1 version of the protocol should it be needed by the system.

To enable HTTP2 for individual VirtualHosts, add the same line within the VirtualHost directive in either httpd.conf or httpd-ssl.conf.

Reload the configuration using the `apachectl`reload command and test the configuration either by using either of the following methods after visiting one of the hosted pages:

```
# grep "HTTP/2.0" /var/log/httpd-access.log
```

This should return something similar to the following:

```
192.168.1.205 - - [18/Oct/2020:18:34:36 -0400] "GET / HTTP/2.0" 304 -
192.0.2.205 - - [18/Oct/2020:19:19:57 -0400] "GET / HTTP/2.0" 304 -
192.0.0.205 - - [18/Oct/2020:19:20:52 -0400] "GET / HTTP/2.0" 304 -
192.0.2.205 - - [18/Oct/2020:19:23:10 -0400] "GET / HTTP/2.0" 304 -
```

The other method is using the web browser's built in site debugger or `tcpdump`; however, using either method is beyond the scope of this document.

Support for HTTP2 reverse proxy connections by using the mod_proxy_http2.so module. When configuring the ProxyPass or RewriteRules [P] statements, they should use h2:// for the connection.

## 32.9.4. Dynamic Websites

In addition to mod_perl and mod_php, other languages are available for creating dynamic web content. These include Django and Ruby on Rails.

### 32.9.4.1. Django

Django is a BSD-licensed framework designed to allow developers to write high performance, elegant web applications quickly. It provides an object-relational mapper so that data types are developed as Python objects. A rich dynamic database-access API is provided for those objects without the developer ever having to write SQL. It also provides an extensible template system so that the logic of the application is separated from the HTML presentation.

Django depends on mod_python, and an SQL database engine. In FreeBSD, the www/py-django port automatically installs mod_python and supports the PostgreSQL, MySQL, or SQLite databases, with the default being SQLite. To change the database engine, type `make config` within /usr/ports/www/py-django, then install the port.

Once Django is installed, the application will need a project directory along with the Apache configuration in order to use the embedded Python interpreter. This interpreter is used to call the application for specific URLs on the site.

To configure Apache to pass requests for certain URLs to the web application, add the following to httpd.conf, specifying the full path to the project directory:

```
<Location "/">
```

```
    SetHandler python-program
    PythonPath "['/dir/to/the/django/packages/'] + sys.path"
    PythonHandler django.core.handlers.modpython
    SetEnv DJANGO_SETTINGS_MODULE mysite.settings
    PythonAutoReload On
    PythonDebug On
</Location>
```

Refer to https://docs.djangoproject.com for more information on how to use Django.

#### 32.9.4.2. Ruby on Rails

Ruby on Rails is another open source web framework that provides a full development stack. It is optimized to make web developers more productive and capable of writing powerful applications quickly. On FreeBSD, it can be installed using the www/rubygem-rails package or port.

Refer to http://guides.rubyonrails.org for more information on how to use Ruby on Rails.

# 32.10. File Transfer Protocol (FTP)

The File Transfer Protocol (FTP) provides users with a simple way to transfer files to and from an FTP server. FreeBSD includes FTP server software, ftpd, in the base system.

FreeBSD provides several configuration files for controlling access to the FTP server. This section summarizes these files. Refer to ftpd(8) for more details about the built-in FTP server.

## 32.10.1. Configuration

The most important configuration step is deciding which accounts will be allowed access to the FTP server. A FreeBSD system has a number of system accounts which should not be allowed FTP access. The list of users disallowed any FTP access can be found in /etc/ftpusers. By default, it includes system accounts. Additional users that should not be allowed access to FTP can be added.

In some cases it may be desirable to restrict the access of some users without preventing them completely from using FTP. This can be accomplished be creating /etc/ftpchroot as described in ftpchroot(5). This file lists users and groups subject to FTP access restrictions.

To enable anonymous FTP access to the server, create a user named `ftp` on the FreeBSD system. Users will then be able to log on to the FTP server with a username of `ftp` or `anonymous`. When prompted for the password, any input will be accepted, but by convention, an email address should be used as the password. The FTP server will call chroot(2) when an anonymous user logs in, to restrict access to only the home directory of the `ftp` user.

There are two text files that can be created to specify welcome messages to be displayed to FTP clients. The contents of /etc/ftpwelcome will be displayed to users before they reach the login prompt. After a successful login, the contents of /etc/ftpmotd will be displayed. Note that the path to this file is relative to the login environment, so the contents of ~ftp/etc/ftpmotd would be displayed for anonymous users.

Once the FTP server has been configured, set the appropriate variable in /etc/rc.conf to start the service during boot:

```
ftpd_enable="YES"
```

To start the service now:

```
# service ftpd start
```

Test the connection to the FTP server by typing:

```
% ftp localhost
```

The ftpd daemon uses syslog(3) to log messages. By default, the system log daemon will write messages related to FTP in /var/log/xferlog. The location of the FTP log can be modified by changing the following line in /etc/syslog.conf:

```
ftp.info      /var/log/xferlog
```



Be aware of the potential problems involved with running an anonymous FTP server. In particular, think twice about allowing anonymous users to upload files. It may turn out that the FTP site becomes a forum for the trade of unlicensed commercial software or worse. If anonymous FTP uploads are required, then verify the permissions so that these files cannot be read by other anonymous users until they have been reviewed by an administrator.

## 32.11. File and Print Services for Microsoft® Windows® Clients (Samba)

Samba is a popular open source software package that provides file and print services using the SMB/CIFS protocol. This protocol is built into Microsoft® Windows® systems. It can be added to non-Microsoft® Windows® systems by installing the Samba client libraries. The protocol allows clients to access shared data and printers. These shares can be mapped as a local disk drive and shared printers can be used as if they were local printers.

On FreeBSD, the Samba client libraries can be installed using the net/samba416 port or package. The client provides the ability for a FreeBSD system to access SMB/CIFS shares in a Microsoft® Windows® network.

A FreeBSD system can also be configured to act as a Samba server by installing the same net/samba416 port or package. This allows the administrator to create SMB/CIFS shares on the FreeBSD system which can be accessed by clients running Microsoft® Windows® or the Samba client libraries.

### 32.11.1. Server Configuration

Samba is configured in /usr/local/etc/smb4.conf. This file must be created before Samba can be used.

A simple smb4.conf to share directories and printers with Windows® clients in a workgroup is shown here. For more complex setups involving LDAP or Active Directory, it is easier to use samba-tool(8) to create the initial smb4.conf.

```
[global]
workgroup = WORKGROUP
server string = Samba Server Version %v
netbios name = ExampleMachine
wins support = Yes
security = user
passdb backend = tdbsam

# Example: share /usr/src accessible only to 'developer' user
[src]
path = /usr/src
valid users = developer
writable  = yes
browsable = yes
read only = no
guest ok = no
public = no
create mask = 0666
directory mask = 0755
```

#### 32.11.1.1. Global Settings

Settings that describe the network are added in /usr/local/etc/smb4.conf:

`workgroup`

The name of the workgroup to be served.

`netbios name`

The NetBIOS name by which a Samba server is known. By default, it is the same as the first component of the host's DNS name.

`server string`

The string that will be displayed in the output of `net view` and some other networking tools that seek to display descriptive text about the server.

`wins support`

Whether Samba will act as a WINS server. Do not enable support for WINS on more than one server on the network.

#### 32.11.1.2. Security Settings

The most important settings in /usr/local/etc/smb4.conf are the security model and the backend password format. These directives control the options:

`security`

If the clients use usernames that are the same as their usernames on the FreeBSD machine, user level security should be used. `security = user` is the default security policy and it requires clients to first log on before they can access shared resources.

Refer to smb.conf(5) to learn about other supported settings for the `security` option.

`passdb backend`

Samba has several different backend authentication models. Clients may be authenticated with LDAP, NIS+, an SQL database, or a modified password file. The recommended authentication method, `tdbsam`, is ideal for simple networks and is covered here. For larger or more complex networks, `ldapsam` is recommended. `smbpasswd` was the former default and is now obsolete.

#### 32.11.1.3. Samba Users

FreeBSD user accounts must be mapped to the `SambaSAMAccount` database for Windows® clients to access the share. Map existing FreeBSD user accounts using pdbedit(8):

```
# pdbedit -a -u username
```

This section has only mentioned the most commonly used settings. Refer to the Official Samba Wiki for additional information about the available configuration options.

### 32.11.2. Starting Samba

To enable Samba at boot time, add the following line to /etc/rc.conf:

```
samba_server_enable="YES"
```

To start Samba now:

```
# service samba_server start
Performing sanity check on Samba configuration: OK
Starting nmbd.
Starting smbd.
```

Samba consists of three separate daemons. Both the nmbd and smbd daemons are started by `samba_enable`. If winbind name resolution is also required, set:

```
winbindd_enable="YES"
```

Samba can be stopped at any time by typing:

```
# service samba_server stop
```

Samba is a complex software suite with functionality that allows broad integration with Microsoft® Windows® networks. For more information about functionality beyond the basic configuration described here, refer to https://www.samba.org.

# 32.12. Clock Synchronization with NTP

Over time, a computer's clock is prone to drift. This is problematic as many network services require the computers on a network to share the same accurate time. Accurate time is also needed to ensure that file timestamps stay consistent. The Network Time Protocol (NTP) is one way to provide clock accuracy in a network.

FreeBSD includes ntpd(8) which can be configured to query other NTP servers to synchronize the clock on that machine or to provide time services to other computers in the network.

This section describes how to configure ntpd on FreeBSD. Further documentation can be found in /usr/share/doc/ntp/ in HTML format.

## 32.12.1. NTP Configuration

On FreeBSD, the built-in ntpd can be used to synchronize a system's clock. ntpd is configured using rc.conf(5) variables and /etc/ntp.conf, as detailed in the following sections.

ntpd communicates with its network peers using UDP packets. Any firewalls between the machine and its NTP peers must be configured to allow UDP packets in and out on port 123.

### 32.12.1.1. The /etc/ntp.conf file

ntpd reads /etc/ntp.conf to determine which NTP servers to query. Choosing several NTP servers is recommended in case one of the servers becomes unreachable or its clock proves unreliable. As ntpd receives responses, it favors reliable servers over the less reliable ones. The servers which are queried can be local to the network, provided by an ISP, or selected from an online list of publicly accessible NTP servers. When choosing a public NTP server, select one that is geographically close and review its usage policy. The `pool` configuration keyword selects one or more servers from a pool of servers. An online list of publicly accessible NTP pools is available, organized by geographic area. In addition, FreeBSD provides a project-sponsored pool, `0.freebsd.pool.ntp.org`.

*Example 43. Sample /etc/ntp.conf*

This is a simple example of an ntp.conf file. It can safely be used as-is; it contains the recommended `restrict` options for operation on a publicly-accessible network connection.

```
# Disallow ntpq control/query access.  Allow peers to be added only
# based on pool and server statements in this file.
restrict default limited kod nomodify notrap noquery nopeer
```

```
restrict source  limited kod nomodify notrap noquery

# Allow unrestricted access from localhost for queries and control.
restrict 127.0.0.1
restrict ::1

# Add a specific server.
server ntplocal.example.com iburst

# Add FreeBSD pool servers until 3-6 good servers are available.
tos minclock 3 maxclock 6
pool 0.freebsd.pool.ntp.org iburst

# Use a local leap-seconds file.
leapfile "/var/db/ntpd.leap-seconds.list"
```

The format of this file is described in ntp.conf(5). The descriptions below provide a quick overview of just the keywords used in the sample file above.

By default, an NTP server is accessible to any network host. The `restrict` keyword controls which systems can access the server. Multiple `restrict` entries are supported, each one refining the restrictions given in previous statements. The values shown in the example grant the local system full query and control access, while allowing remote systems only the ability to query the time. For more details, refer to the `Access Control Support` subsection of ntp.conf(5).

The `server` keyword specifies a single server to query. The file can contain multiple server keywords, with one server listed on each line. The `pool` keyword specifies a pool of servers. ntpd will add one or more servers from this pool as needed to reach the number of peers specified using the `tos minclock` value. The `iburst` keyword directs ntpd to perform a burst of eight quick packet exchanges with a server when contact is first established, to help quickly synchronize system time.

The `leapfile` keyword specifies the location of a file containing information about leap seconds. The file is updated automatically by periodic(8). The file location specified by this keyword must match the location set in the `ntp_db_leapfile` variable in /etc/rc.conf.

#### 32.12.1.2. NTP entries in /etc/rc.conf

Set `ntpd_enable=YES` to start ntpd at boot time. Once `ntpd_enable=YES` has been added to /etc/rc.conf, ntpd can be started immediately without rebooting the system by typing:

```
# service ntpd start
```

Only `ntpd_enable` must be set to use ntpd. The rc.conf variables listed below may also be set as needed.

Set `ntpd_sync_on_start=YES` to allow ntpd to step the clock any amount, one time at startup. Normally ntpd will log an error message and exit if the clock is off by more than 1000 seconds. This option is especially useful on systems without a battery-backed realtime clock.

Set `ntpd_oomprotect=YES` to protect the ntpd daemon from being killed by the system attempting to recover from an Out Of Memory (OOM) condition.

Set `ntpd_config=` to the location of an alternate ntp.conf file.

Set `ntpd_flags=` to contain any other ntpd flags as needed, but avoid using these flags which are managed internally by /etc/rc.d/ntpd:

- `-p` (pid file location)
- `-c` (set `ntpd_config=` instead)

#### 32.12.1.3. ntpd and the unprivileged `ntpd` user

ntpd on FreeBSD can start and run as an unprivileged user. Doing so requires the mac_ntpd(4) policy module. The /etc/rc.d/ntpd startup script first examines the NTP configuration. If possible, it loads the `mac_ntpd` module, then starts ntpd as unprivileged user `ntpd` (user id 123). To avoid problems with file and directory access, the startup script will not automatically start ntpd as `ntpd` when the configuration contains any file-related options.

The presence of any of the following in `ntpd_flags` requires manual configuration as described below to run as the `ntpd` user:

- -f or --driftfile
- -i or --jaildir
- -k or --keyfile
- -l or --logfile
- -s or --statsdir

The presence of any of the following keywords in ntp.conf requires manual configuration as described below to run as the `ntpd` user:

- crypto
- driftfile
- key
- logdir
- statsdir

To manually configure ntpd to run as user `ntpd`:

- Ensure that the `ntpd` user has access to all the files and directories specified in the configuration.
- Arrange for the `mac_ntpd` module to be loaded or compiled into the kernel. See mac_ntpd(4) for details.
- Set `ntpd_user="ntpd"` in /etc/rc.conf

### 32.12.2. Using NTP with a PPP Connection

ntpd does not need a permanent connection to the Internet to function properly. However, if a PPP connection is configured to dial out on demand, NTP traffic should be prevented from triggering a dial out or keeping the connection alive. This can be configured with `filter` directives in /etc/ppp/ppp.conf. For example:

```
set filter dial 0 deny udp src eq 123
# Prevent NTP traffic from initiating dial out
set filter dial 1 permit 0 0
set filter alive 0 deny udp src eq 123
# Prevent incoming NTP traffic from keeping the connection open
set filter alive 1 deny udp dst eq 123
# Prevent outgoing NTP traffic from keeping the connection open
set filter alive 2 permit 0/0 0/0
```

For more details, refer to the `PACKET FILTERING` section in ppp(8) and the examples in /usr/share/examples/ppp/.



Some Internet access providers block low-numbered ports, preventing NTP from functioning since replies never reach the machine.

## 32.13. iSCSI Initiator and Target Configuration

iSCSI is a way to share storage over a network. Unlike NFS, which works at the file system level, iSCSI works at the block device level.

In iSCSI terminology, the system that shares the storage is known as the *target*. The storage can be a physical disk, or an area representing multiple disks or a portion of a physical disk. For example, if the disk(s) are formatted with ZFS, a zvol can be created to use as the iSCSI storage.

The clients which access the iSCSI storage are called *initiators*. To initiators, the storage available through iSCSI appears as a raw, unformatted disk known as a LUN. Device nodes for the disk appear in /dev/ and the device must be separately formatted and mounted.

FreeBSD provides a native, kernel-based iSCSI target and initiator. This section describes how to configure a FreeBSD system as a target or an initiator.

### 32.13.1. Configuring an iSCSI Target

To configure an iSCSI target, create the /etc/ctl.conf configuration file, add a line to /etc/rc.conf to make sure the ctld(8) daemon is automatically started at boot, and then start the daemon.

The following is an example of a simple /etc/ctl.conf configuration file. Refer to ctl.conf(5) for a complete description of this file's available options.

```
portal-group pg0 {
    discovery-auth-group no-authentication
```

```
    listen 0.0.0.0
    listen [::]
}

target iqn.2012-06.com.example:target0 {
    auth-group no-authentication
    portal-group pg0

    lun 0 {
        path /data/target0-0
        size 4G
    }
}
```

The first entry defines the `pg0` portal group. Portal groups define which network addresses the ctld(8) daemon will listen on. The `discovery-auth-group no-authentication` entry indicates that any initiator is allowed to perform iSCSI target discovery without authentication. Lines three and four configure ctld(8) to listen on all IPv4 (`listen 0.0.0.0`) and IPv6 (`listen [::]`) addresses on the default port of 3260.

It is not necessary to define a portal group as there is a built-in portal group called `default`. In this case, the difference between `default` and `pg0` is that with `default`, target discovery is always denied, while with `pg0`, it is always allowed.

The second entry defines a single target. Target has two possible meanings: a machine serving iSCSI or a named group of LUNs. This example uses the latter meaning, where `iqn.2012-06.com.example:target0` is the target name. This target name is suitable for testing purposes. For actual use, change `com.example` to the real domain name, reversed. The `2012-06` represents the year and month of acquiring control of that domain name, and `target0` can be any value. Any number of targets can be defined in this configuration file.

The `auth-group no-authentication` line allows all initiators to connect to the specified target and `portal-group pg0` makes the target reachable through the `pg0` portal group.

The next section defines the LUN. To the initiator, each LUN will be visible as a separate disk device. Multiple LUNs can be defined for each target. Each LUN is identified by a number, where LUN 0 is mandatory. The `path /data/target0-0` line defines the full path to a file or zvol backing the LUN. That path must exist before starting ctld(8). The second line is optional and specifies the size of the LUN.

Next, to make sure the ctld(8) daemon is started at boot, add this line to /etc/rc.conf:

```
ctld_enable="YES"
```

To start ctld(8) now, run this command:

```
# service ctld start
```

As the ctld(8) daemon is started, it reads /etc/ctl.conf. If this file is edited after the daemon starts, use this command so that the changes take effect immediately:

```
# service ctld reload
```

#### 32.13.1.1. Authentication

The previous example is inherently insecure as it uses no authentication, granting anyone full access to all targets. To require a username and password to access targets, modify the configuration as follows:

```
auth-group ag0 {
    chap username1 secretsecret
    chap username2 anothersecret
}

portal-group pg0 {
    discovery-auth-group no-authentication
    listen 0.0.0.0
    listen [::]
}

target iqn.2012-06.com.example:target0 {
    auth-group ag0
    portal-group pg0
    lun 0 {
        path /data/target0-0
        size 4G
    }
}
```

The `auth-group` section defines username and password pairs. An initiator trying to connect to `iqn.2012-06.com.example:target0` must first specify a defined username and secret. However, target discovery is still permitted without authentication. To require target discovery authentication, set `discovery-auth-group` to a defined `auth-group` name instead of `no-authentication`.

It is common to define a single exported target for every initiator. As a shorthand for the syntax above, the username and password can be specified directly in the target entry:

```
target iqn.2012-06.com.example:target0 {
    portal-group pg0
    chap username1 secretsecret

    lun 0 {
        path /data/target0-0
        size 4G
    }
```

```
}
```

### 32.13.2. Configuring an iSCSI Initiator



The iSCSI initiator described in this section is supported starting with FreeBSD 10.0-RELEASE. To use the iSCSI initiator available in older versions, refer to iscontrol(8).

The iSCSI initiator requires that the iscsid(8) daemon is running. This daemon does not use a configuration file. To start it automatically at boot, add this line to /etc/rc.conf:

```
iscsid_enable="YES"
```

To start iscsid(8) now, run this command:

```
# service iscsid start
```

Connecting to a target can be done with or without an /etc/iscsi.conf configuration file. This section demonstrates both types of connections.

#### 32.13.2.1. Connecting to a Target Without a Configuration File

To connect an initiator to a single target, specify the IP address of the portal and the name of the target:

```
# iscsictl -A -p 10.10.10.10 -t iqn.2012-06.com.example:target0
```

To verify if the connection succeeded, run `iscsictl` without any arguments. The output should look similar to this:

```
Target name                                    Target portal   State
iqn.2012-06.com.example:target0                10.10.10.10     Connected: da0
```

In this example, the iSCSI session was successfully established, with /dev/da0 representing the attached LUN. If the `iqn.2012-06.com.example:target0` target exports more than one LUN, multiple device nodes will be shown in that section of the output:

```
Connected: da0 da1 da2.
```

Any errors will be reported in the output, as well as the system logs. For example, this message usually means that the iscsid(8) daemon is not running:

```
Target name                                    Target portal   State
```

```
iqn.2012-06.com.example:target0                 10.10.10.10     Waiting for iscsid(8)
```

The following message suggests a networking problem, such as a wrong IP address or port:

```
Target name                                     Target portal   State
iqn.2012-06.com.example:target0                 10.10.10.11     Connection refused
```

This message means that the specified target name is wrong:

```
Target name                                     Target portal   State
iqn.2012-06.com.example:target0                 10.10.10.10     Not found
```

This message means that the target requires authentication:

```
Target name                                     Target portal   State
iqn.2012-06.com.example:target0                 10.10.10.10     Authentication failed
```

To specify a CHAP username and secret, use this syntax:

```
# iscsictl -A -p 10.10.10.10 -t iqn.2012-06.com.example:target0 -u user -s
secretsecret
```

#### 32.13.2.2. Connecting to a Target with a Configuration File

To connect using a configuration file, create /etc/iscsi.conf with contents like this:

```
t0 {
    TargetAddress   = 10.10.10.10
    TargetName      = iqn.2012-06.com.example:target0
    AuthMethod      = CHAP
    chapIName       = user
    chapSecret      = secretsecret
}
```

The `t0` specifies a nickname for the configuration file section. It will be used by the initiator to specify which configuration to use. The other lines specify the parameters to use during connection. The `TargetAddress` and `TargetName` are mandatory, whereas the other options are optional. In this example, the CHAP username and secret are shown.

To connect to the defined target, specify the nickname:

```
# iscsictl -An t0
```

Alternately, to connect to all targets defined in the configuration file, use:

```
# iscsictl -Aa
```

To make the initiator automatically connect to all targets in /etc/iscsi.conf, add the following to /etc/rc.conf:

```
iscsictl_enable="YES"
iscsictl_flags="-Aa"
```

# Chapter 33. Firewalls

## 33.1. Synopsis

Firewalls make it possible to filter the incoming and outgoing traffic that flows through a system. A firewall can use one or more sets of "rules" to inspect network packets as they come in or go out of network connections and either allows the traffic through or blocks it. The rules of a firewall can inspect one or more characteristics of the packets such as the protocol type, source or destination host address, and source or destination port.

Firewalls can enhance the security of a host or a network. They can be used to do one or more of the following:

- Protect and insulate the applications, services, and machines of an internal network from unwanted traffic from the public Internet.
- Limit or disable access from hosts of the internal network to services of the public Internet.
- Support network address translation (NAT), which allows an internal network to use private IP addresses and share a single connection to the public Internet using either a single IP address or a shared pool of automatically assigned public addresses.

FreeBSD has three firewalls built into the base system: PF, IPFW, and IPFILTER, also known as IPF. FreeBSD also provides two traffic shapers for controlling bandwidth usage: altq(4) and dummynet(4). ALTQ has traditionally been closely tied with PF and dummynet with IPFW. Each firewall uses rules to control the access of packets to and from a FreeBSD system, although they go about it in different ways and each has a different rule syntax.

FreeBSD provides multiple firewalls in order to meet the different requirements and preferences for a wide variety of users. Each user should evaluate which firewall best meets their needs.

Read this chapter to learn:

- How to define packet filtering rules.
- The differences between the firewalls built into FreeBSD.
- How to use and configure the PF firewall.
- How to use and configure the IPFW firewall.
- How to use and configure the IPFILTER firewall.

Before reading this chapter:

- Understand basic FreeBSD and Internet concepts.



Since all firewalls are based on inspecting the values of selected packet control fields, the creator of the firewall ruleset must have an understanding of how TCP/IP works, what the different values in the packet control fields are, and how these values are used in a normal session conversation. For a good introduction, refer to Daryl's TCP/IP Primer.

## 33.2. Firewall Concepts

A ruleset contains a group of rules which pass or block packets based on the values contained in the packet. The bi-directional exchange of packets between hosts comprises a session conversation. The firewall ruleset processes both the packets arriving from the public Internet, as well as the packets produced by the system as a response to them. Each TCP/IP service is predefined by its protocol and listening port. Packets destined for a specific service originate from the source address using an unprivileged port and target the specific service port on the destination address. All the above parameters can be used as selection criteria to create rules which will pass or block services.

To lookup unknown port numbers, refer to /etc/services. Alternatively, visit https://en.wikipedia.org/wiki/List_of_TCP_and_UDP_port_numbers and do a port number lookup to find the purpose of a particular port number.

Check out this link for port numbers used by Trojans.

FTP has two modes: active mode and passive mode. The difference is in how the data channel is acquired. Passive mode is more secure as the data channel is acquired by the ordinal ftp session requester. For a good explanation of FTP and the different modes, see http://www.slacksite.com/other/ftp.html.

A firewall ruleset can be either "exclusive" or "inclusive". An exclusive firewall allows all traffic through except for the traffic matching the ruleset. An inclusive firewall does the reverse as it only allows traffic matching the rules through and blocks everything else.

An inclusive firewall offers better control of the outgoing traffic, making it a better choice for systems that offer services to the public Internet. It also controls the type of traffic originating from the public Internet that can gain access to a private network. All traffic that does not match the rules is blocked and logged. Inclusive firewalls are generally safer than exclusive firewalls because they significantly reduce the risk of allowing unwanted traffic.



Unless noted otherwise, all configuration and example rulesets in this chapter create inclusive firewall rulesets.

Security can be tightened further using a "stateful firewall". This type of firewall keeps track of open connections and only allows traffic which either matches an existing connection or opens a new, allowed connection.

Stateful filtering treats traffic as a bi-directional exchange of packets comprising a session. When state is specified on a matching rule the firewall dynamically generates internal rules for each anticipated packet being exchanged during the session. It has sufficient matching capabilities to determine if a packet is valid for a session. Any packets that do not properly fit the session template are automatically rejected.

When the session completes, it is removed from the dynamic state table.

Stateful filtering allows one to focus on blocking/passing new sessions. If the new session is passed, all its subsequent packets are allowed automatically and any impostor packets are automatically rejected. If a new session is blocked, none of its subsequent packets are allowed. Stateful filtering

provides advanced matching abilities capable of defending against the flood of different attack methods employed by attackers.

NAT stands for *Network Address Translation*. NAT function enables the private LAN behind the firewall to share a single ISP-assigned IP address, even if that address is dynamically assigned. NAT allows each computer in the LAN to have Internet access, without having to pay the ISP for multiple Internet accounts or IP addresses.

NAT will automatically translate the private LAN IP address for each system on the LAN to the single public IP address as packets exit the firewall bound for the public Internet. It also performs the reverse translation for returning packets.

According to RFC 1918, the following IP address ranges are reserved for private networks which will never be routed directly to the public Internet, and therefore are available for use with NAT:

- `10.0.0.0/8`.
- `172.16.0.0/12`.
- `192.168.0.0/16`.



When working with the firewall rules, be *very careful*. Some configurations *can lock the administrator out* of the server. To be on the safe side, consider performing the initial firewall configuration from the local console rather than doing it remotely over ssh.

## 33.3. PF

Since FreeBSD 5.3, a ported version of OpenBSD's PF firewall has been included as an integrated part of the base system. PF is a complete, full-featured firewall that has optional support for ALTQ (Alternate Queuing), which provides Quality of Service (QoS).

The OpenBSD Project maintains the definitive reference for PF in the PF FAQ. Peter Hansteen maintains a thorough PF tutorial at http://home.nuug.no/~peter/pf/.



When reading the PF FAQ, keep in mind that FreeBSD's version of PF has diverged substantially from the upstream OpenBSD version over the years. Not all features work the same way on FreeBSD as they do in OpenBSD and vice versa.

The FreeBSD packet filter mailing list is a good place to ask questions about configuring and running the PF firewall. Check the mailing list archives before asking a question as it may have already been answered.

This section of the Handbook focuses on PF as it pertains to FreeBSD. It demonstrates how to enable PF and ALTQ. It also provides several examples for creating rulesets on a FreeBSD system.

### 33.3.1. Enabling PF

To use PF, its kernel module must be first loaded. This section describes the entries that can be added to /etc/rc.conf to enable PF.

Start by adding `pf_enable=yes` to /etc/rc.conf:

```
# sysrc pf_enable=yes
```

Additional options, described in pfctl(8), can be passed to PF when it is started. Add or change this entry in /etc/rc.conf and specify any required flags between the two quotes (""):

```
pf_flags=""                   # additional flags for pfctl startup
```

PF will not start if it cannot find its ruleset configuration file. By default, FreeBSD does not ship with a ruleset and there is no /etc/pf.conf. Example rulesets can be found in /usr/share/examples/pf/. If a custom ruleset has been saved somewhere else, add a line to /etc/rc.conf which specifies the full path to the file:

```
pf_rules="/path/to/pf.conf"
```

Logging support for PF is provided by pflog(4). To enable logging support, add `pflog_enable=yes` to /etc/rc.conf:

```
# sysrc pflog_enable=yes
```

The following lines can also be added to change the default location of the log file or to specify any additional flags to pass to pflog(4) when it is started:

```
pflog_logfile="/var/log/pflog"  # where pflogd should store the logfile
pflog_flags=""                  # additional flags for pflogd startup
```

Finally, if there is a LAN behind the firewall and packets need to be forwarded for the computers on the LAN, or NAT is required, enable the following option:

```
gateway_enable="YES"            # Enable as LAN gateway
```

After saving the needed edits, PF can be started with logging support by typing:

```
# service pf start
# service pflog start
```

By default, PF reads its configuration rules from /etc/pf.conf and modifies, drops, or passes packets according to the rules or definitions specified in this file. The FreeBSD installation includes several sample files located in /usr/share/examples/pf/. Refer to the PF FAQ for complete coverage of PF rulesets.

To control PF, use `pfctl`. Useful `pfctl` Options summarizes some useful options to this command. Refer to pfctl(8) for a description of all available options:

*Table 49. Useful `pfctl` Options*

| Command | Purpose |
|---|---|
| `pfctl -e` | Enable PF. |
| `pfctl -d` | Disable PF. |
| `pfctl -F all -f /etc/pf.conf` | Flush all NAT, filter, state, and table rules and reload /etc/pf.conf. |
| `pfctl -s [ rules \| nat \| states ]` | Report on the filter rules, NAT rules, or state table. |
| `pfctl -vnf /etc/pf.conf` | Check /etc/pf.conf for errors, but do not load ruleset. |



security/sudo is useful for running commands like `pfctl` that require elevated privileges. It can be installed from the Ports Collection.

To keep an eye on the traffic that passes through the PF firewall, consider installing the sysutils/pftop package or port. Once installed, pftop can be run to view a running snapshot of traffic in a format which is similar to top(1).

### 33.3.2. PF Rulesets

This section demonstrates how to create a customized ruleset. It starts with the simplest of rulesets and builds upon its concepts using several examples to demonstrate real-world usage of PF's many features.

The simplest possible ruleset is for a single machine that does not run any services and which needs access to one network, which may be the Internet. To create this minimal ruleset, edit /etc/pf.conf so it looks like this:

```
block in all
pass out all keep state
```

The first rule denies all incoming traffic by default. The second rule allows connections created by this system to pass out, while retaining state information on those connections. This state information allows return traffic for those connections to pass back and should only be used on machines that can be trusted. The ruleset can be loaded with:

```
# pfctl -e ; pfctl -f /etc/pf.conf
```

In addition to keeping state, PF provides *lists* and *macros* which can be defined for use when creating rules. Macros can include lists and need to be defined before use. As an example, insert these lines at the very top of the ruleset:

```
tcp_services = "{ ssh, smtp, domain, www, pop3, auth, pop3s }"
udp_services = "{ domain }"
```

PF understands port names as well as port numbers, as long as the names are listed in /etc/services. This example creates two macros. The first is a list of seven TCP port names and the second is one UDP port name. Once defined, macros can be used in rules. In this example, all traffic is blocked except for the connections initiated by this system for the seven specified TCP services and the one specified UDP service:

```
tcp_services = "{ ssh, smtp, domain, www, pop3, auth, pop3s }"
udp_services = "{ domain }"
block all
pass out proto tcp to any port $tcp_services keep state
pass proto udp to any port $udp_services keep state
```

Even though UDP is considered to be a stateless protocol, PF is able to track some state information. For example, when a UDP request is passed which asks a name server about a domain name, PF will watch for the response to pass it back.

Whenever an edit is made to a ruleset, the new rules must be loaded so they can be used:

```
# pfctl -f /etc/pf.conf
```

If there are no syntax errors, `pfctl` will not output any messages during the rule load. Rules can also be tested before attempting to load them:

```
# pfctl -nf /etc/pf.conf
```

Including `-n` causes the rules to be interpreted only, but not loaded. This provides an opportunity to correct any errors. At all times, the last valid ruleset loaded will be enforced until either PF is disabled or a new ruleset is loaded.



Adding `-v` to a `pfctl` ruleset verify or load will display the fully parsed rules exactly the way they will be loaded. This is extremely useful when debugging rules.

#### 33.3.2.1. A Simple Gateway with NAT

This section demonstrates how to configure a FreeBSD system running PF to act as a gateway for at least one other machine. The gateway needs at least two network interfaces, each connected to a separate network. In this example, xl0 is connected to the Internet and xl1 is connected to the internal network.

First, enable the gateway to let the machine forward the network traffic it receives on one interface to another interface. This sysctl setting will forward IPv4 packets:

```
# sysctl net.inet.ip.forwarding=1
```

To forward IPv6 traffic, use:

```
# sysctl net.inet6.ip6.forwarding=1
```

To enable these settings at system boot, use sysrc(8) to add them to /etc/rc.conf:

```
# sysrc gateway_enable=yes
# sysrc ipv6_gateway_enable=yes
```

Verify with `ifconfig` that both of the interfaces are up and running.

Next, create the PF rules to allow the gateway to pass traffic. While the following rule allows stateful traffic from hosts of the internal network to pass to the gateway, the `to` keyword does not guarantee passage all the way from source to destination:

```
pass in on xl1 from xl1:network to xl0:network port $ports keep state
```

That rule only lets the traffic pass in to the gateway on the internal interface. To let the packets go further, a matching rule is needed:

```
pass out on xl0 from xl1:network to xl0:network port $ports keep state
```

While these two rules will work, rules this specific are rarely needed. For a busy network admin, a readable ruleset is a safer ruleset. The remainder of this section demonstrates how to keep the rules as simple as possible for readability. For example, those two rules could be replaced with one rule:

```
pass from xl1:network to any port $ports keep state
```

The `interface:network` notation can be replaced with a macro to make the ruleset even more readable. For example, a `$localnet` macro could be defined as the network directly attached to the internal interface (`$xl1:network`). Alternatively, the definition of `$localnet` could be changed to an *IP address/netmask* notation to denote a network, such as `192.168.100.1/24` for a subnet of private addresses.

If required, `$localnet` could even be defined as a list of networks. Whatever the specific needs, a sensible `$localnet` definition could be used in a typical pass rule as follows:

```
pass from $localnet to any port $ports keep state
```

The following sample ruleset allows all traffic initiated by machines on the internal network. It first

defines two macros to represent the external and internal 3COM interfaces of the gateway.



For dialup users, the external interface will use tun0. For an ADSL connection, specifically those using PPP over Ethernet (PPPoE), the correct external interface is tun0, not the physical Ethernet interface.

```
ext_if = "xl0"  # macro for external interface - use tun0 for PPPoE
int_if = "xl1"  # macro for internal interface
localnet = $int_if:network
# ext_if IP address could be dynamic, hence ($ext_if)
nat on $ext_if from $localnet to any -> ($ext_if)
block all
pass from { lo0, $localnet } to any keep state
```

This ruleset introduces the `nat` rule which is used to handle the network address translation from the non-routable addresses inside the internal network to the IP address assigned to the external interface. The parentheses surrounding the last part of the nat rule `($ext_if)` is included when the IP address of the external interface is dynamically assigned. It ensures that network traffic runs without serious interruptions even if the external IP address changes.

Note that this ruleset probably allows more traffic to pass out of the network than is needed. One reasonable setup could create this macro:

```
client_out = "{ ftp-data, ftp, ssh, domain, pop3, auth, nntp, http, \
    https, cvspserver, 2628, 5999, 8000, 8080 }"
```

to use in the main pass rule:

```
pass inet proto tcp from $localnet to any port $client_out \
    flags S/SA keep state
```

A few other pass rules may be needed. This one enables SSH on the external interface:

```
pass in inet proto tcp to $ext_if port ssh
```

This macro definition and rule allows DNS and NTP for internal clients:

```
udp_services = "{ domain, ntp }"
pass quick inet proto { tcp, udp } to any port $udp_services keep state
```

Note the `quick` keyword in this rule. Since the ruleset consists of several rules, it is important to understand the relationships between the rules in a ruleset. Rules are evaluated from top to bottom, in the sequence they are written. For each packet or connection evaluated by PF, *the last matching rule* in the ruleset is the one which is applied. However, when a packet matches a rule

which contains the `quick` keyword, the rule processing stops and the packet is treated according to that rule. This is very useful when an exception to the general rules is needed.

#### 33.3.2.2. Creating an FTP Proxy

Configuring working FTP rules can be problematic due to the nature of the FTP protocol. FTP predates firewalls by several decades and is insecure in its design. The most common points against using FTP include:

- Passwords are transferred in the clear.
- The protocol demands the use of at least two TCP connections (control and data) on separate ports.
- When a session is established, data is communicated using randomly selected ports.

All of these points present security challenges, even before considering any potential security weaknesses in client or server software. More secure alternatives for file transfer exist, such as sftp(1) or scp(1), which both feature authentication and data transfer over encrypted connections.

For those situations when FTP is required, PF provides redirection of FTP traffic to a small proxy program called ftp-proxy(8), which is included in the base system of FreeBSD. The role of the proxy is to dynamically insert and delete rules in the ruleset, using a set of anchors, to correctly handle FTP traffic.

To enable the FTP proxy, add this line to /etc/rc.conf:

```
ftpproxy_enable="YES"
```

Then start the proxy by running:

```
# service ftp-proxy start
```

For a basic configuration, three elements need to be added to /etc/pf.conf. First, the anchors which the proxy will use to insert the rules it generates for the FTP sessions:

```
nat-anchor "ftp-proxy/*"
rdr-anchor "ftp-proxy/*"
```

Second, a pass rule is needed to allow FTP traffic in to the proxy.

Third, redirection and NAT rules need to be defined before the filtering rules. Insert this `rdr` rule immediately after the `nat` rule:

```
rdr pass on $int_if proto tcp from any to any port ftp -> 127.0.0.1 port 8021
```

Finally, allow the redirected traffic to pass:

```
pass out proto tcp from $proxy to any port ftp
```

where `$proxy` expands to the address the proxy daemon is bound to.

Save /etc/pf.conf, load the new rules, and verify from a client that FTP connections are working:

```
# pfctl -f /etc/pf.conf
```

This example covers a basic setup where the clients in the local network need to contact FTP servers elsewhere. This basic configuration should work well with most combinations of FTP clients and servers. As shown in ftp-proxy(8), the proxy's behavior can be changed in various ways by adding options to the `ftpproxy_flags=` line. Some clients or servers may have specific quirks that must be compensated for in the configuration, or there may be a need to integrate the proxy in specific ways such as assigning FTP traffic to a specific queue.

For ways to run an FTP server protected by PF and ftp-proxy(8), configure a separate `ftp-proxy` in reverse mode, using `-R`, on a separate port with its own redirecting pass rule.

#### 33.3.2.3. Managing ICMP

Many of the tools used for debugging or troubleshooting a TCP/IP network rely on the Internet Control Message Protocol (ICMP), which was designed specifically with debugging in mind.

The ICMP protocol sends and receives *control messages* between hosts and gateways, mainly to provide feedback to a sender about any unusual or difficult conditions enroute to the target host. Routers use ICMP to negotiate packet sizes and other transmission parameters in a process often referred to as *path MTU discovery*.

From a firewall perspective, some ICMP control messages are vulnerable to known attack vectors. Also, letting all diagnostic traffic pass unconditionally makes debugging easier, but it also makes it easier for others to extract information about the network. For these reasons, the following rule may not be optimal:

```
pass inet proto icmp from any to any
```

One solution is to let all ICMP traffic from the local network through while stopping all probes from outside the network:

```
pass inet proto icmp from $localnet to any keep state
pass inet proto icmp from any to $ext_if keep state
```

Additional options are available which demonstrate some of PF's flexibility. For example, rather than allowing all ICMP messages, one can specify the messages used by ping(8) and traceroute(8). Start by defining a macro for that type of message:

```
icmp_types = "echoreq"
```

and a rule which uses the macro:

```
pass inet proto icmp all icmp-type $icmp_types keep state
```

If other types of ICMP packets are needed, expand `icmp_types` to a list of those packet types. Type `more /usr/src/sbin/pfctl/pfctl_parser.c` to see the list of ICMP message types supported by PF. Refer to http://www.iana.org/assignments/icmp-parameters/icmp-parameters.xhtml for an explanation of each message type.

Since Unix `traceroute` uses UDP by default, another rule is needed to allow Unix `traceroute`:

```
# allow out the default range for traceroute(8):
pass out on $ext_if inet proto udp from any to any port 33433 >< 33626 keep state
```

Since `TRACERT.EXE` on Microsoft Windows systems uses ICMP echo request messages, only the first rule is needed to allow network traces from those systems. Unix `traceroute` can be instructed to use other protocols as well, and will use ICMP echo request messages if `-I` is used. Check the traceroute(8) man page for details.

#### 33.3.2.3.1. Path MTU Discovery

Internet protocols are designed to be device independent, and one consequence of device independence is that the optimal packet size for a given connection cannot always be predicted reliably. The main constraint on packet size is the *Maximum Transmission Unit* (MTU) which sets the upper limit on the packet size for an interface. Type `ifconfig` to view the MTUs for a system's network interfaces.

TCP/IP uses a process known as path MTU discovery to determine the right packet size for a connection. This process sends packets of varying sizes with the "Do not fragment" flag set, expecting an ICMP return packet of "type 3, code 4" when the upper limit has been reached. Type 3 means "destination unreachable", and code 4 is short for "fragmentation needed, but the do-not-fragment flag is set". To allow path MTU discovery in order to support connections to other MTUs, add the `destination unreachable` type to the `icmp_types` macro:

```
icmp_types = "{ echoreq, unreach }"
```

Since the pass rule already uses that macro, it does not need to be modified to support the new ICMP type:

```
pass inet proto icmp all icmp-type $icmp_types keep state
```

PF allows filtering on all variations of ICMP types and codes. The list of possible types and codes are